Introduction to Artificial Intelligence and Machine Learning

IE 206
Multi-layer Perceptrons

March 6, 2026.

1. Perceptron: Quick Recap
2. Moving on from Perceptron
3. Multi Layer Perceptron
   - MLP-Data Perspective

Perceptron - Data Perspective

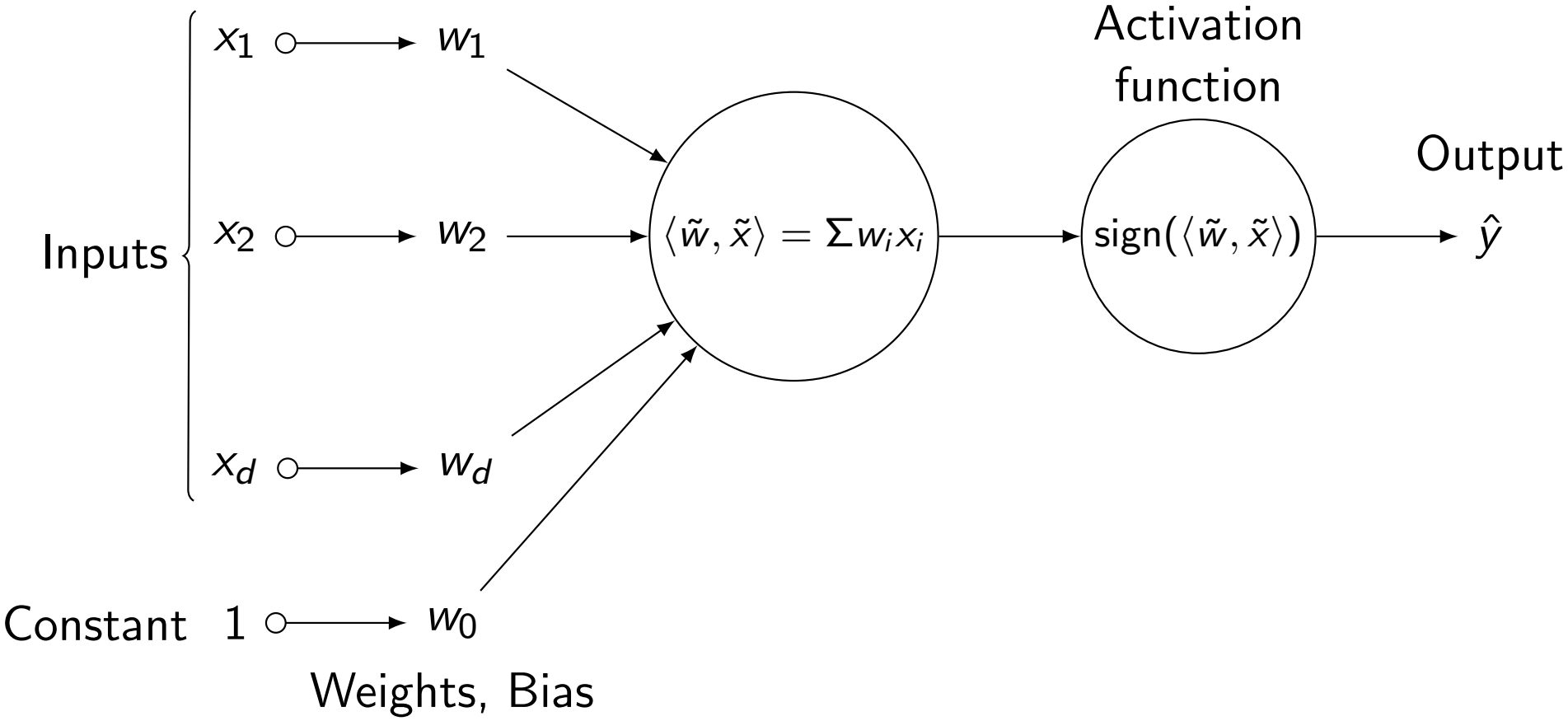


Figure: Perceptron unit

Perceptron - Training

Perceptron Training Procedure

$\tilde{w}^1 = 0$
for $t \leftarrow 1, 2, 3, \ldots$ **do**
    receive (x^t, y^t), $x^t \in \mathbb{R}^d$, $y^t \in \{+1, -1\}$.
    Transform x^t into $\tilde{x}^t = (x^t, 1) \in \mathbb{R}^{d+1}$.
    $\widehat{y} = Perceptron(\tilde{x}^t; \tilde{w}^t)$
    if $\widehat{y} \neq y^t$ **then**
        $\tilde{w}^{t+1} = \tilde{w}^t + y^t \tilde{x}^t$
    else
        $\tilde{w}^{t+1} = \tilde{w}^t$

Perceptron Convergence - Separability Assumption

Linear Separability Assumption

Let $D = \{(x^t, y^t)\}_{t=1}^{\infty}$ denote the training data where $x^t \in \mathbb{R}^d$, $y^t \in \{+1, -1\}$, $\forall t = 1, 2, \ldots$. Then there exist $\mathbb{R}^d \ni w^* \neq 0,\ \gamma > 0$, such that:

$$\langle w^*, x^t \rangle > \gamma \text{ where } y^t = 1,$$
$$\langle w^*, x^t \rangle < -\gamma \text{ where } y^t = -1.$$

Perceptron - Caveat

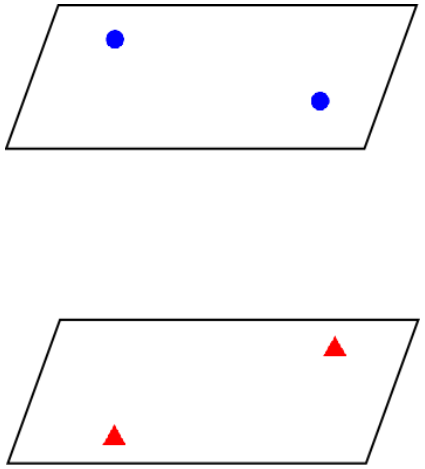
- Not suitable when **linear separability assumption** fails
- Example: Classical XOR problem

Perceptron - Caveat

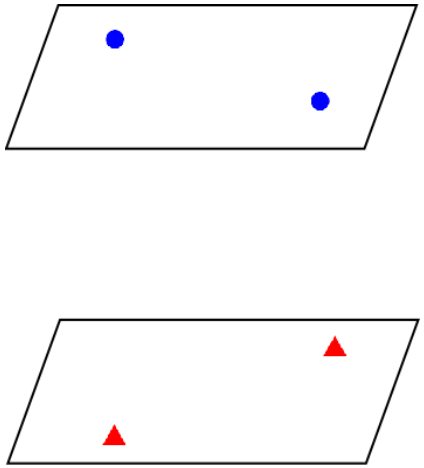
- Not suitable when **linear separability assumption** fails
- Example: Classical XOR problem

x_1	x_2	$y = x_1 \oplus x_2$
0	0	-1
0	1	1
1	0	1
1	1	-1

Moving away from perceptron - Dealing with XOR problem

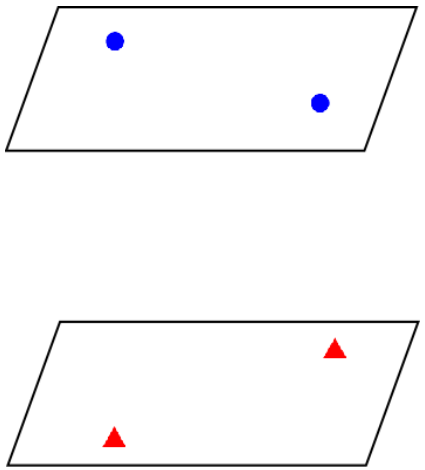


Moving away from perceptron - Dealing with XOR problem



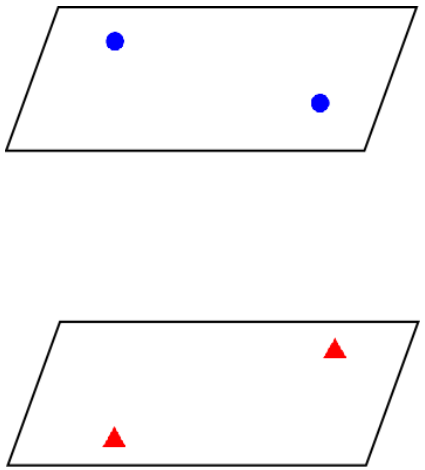
- Assume that the sample features $x \in \mathbb{R}^d$.

Moving away from perceptron - Dealing with XOR problem



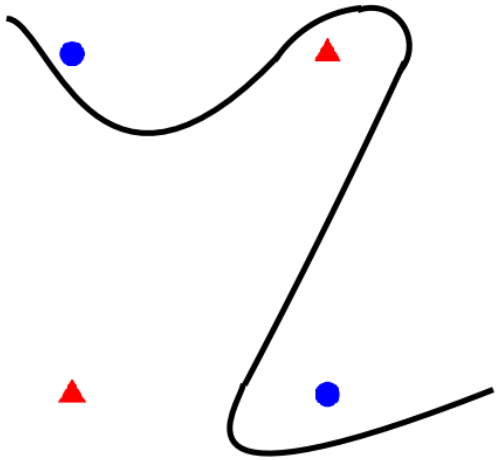
- Assume that the sample features $x \in \mathbb{R}^d$.
- **Idea:** Use a transformation $\psi : \mathbb{R}^d \to \mathbb{R}^q$, where $q \gg d$, to lift the data samples $x \in \mathbb{R}^d$ into $\psi(x) \in \mathbb{R}^q$ hoping to see a separating hyperplane in the transformed space.

Moving away from perceptron - Dealing with XOR problem

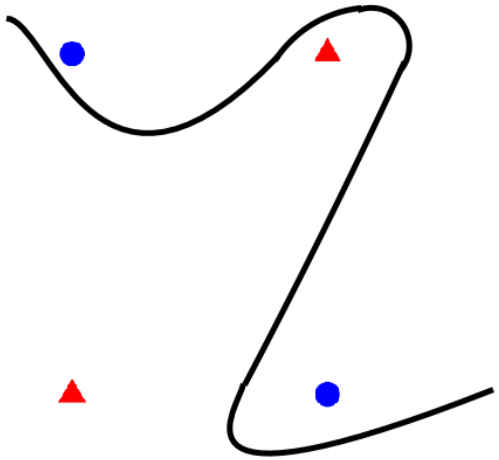


- Assume that the sample features $x \in \mathbb{R}^d$.
- **Idea:** Use a transformation $\psi : \mathbb{R}^d \to \mathbb{R}^q$, where $q \gg d$, to lift the data samples $x \in \mathbb{R}^d$ into $\psi(x) \in \mathbb{R}^q$ hoping to see a separating hyperplane in the transformed space.
- Forms the core idea behind kernel methods.

Moving away from perceptron - Dealing with XOR problem

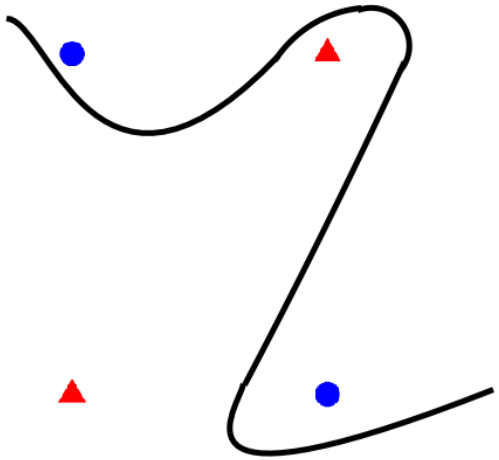


Moving away from perceptron - Dealing with XOR problem



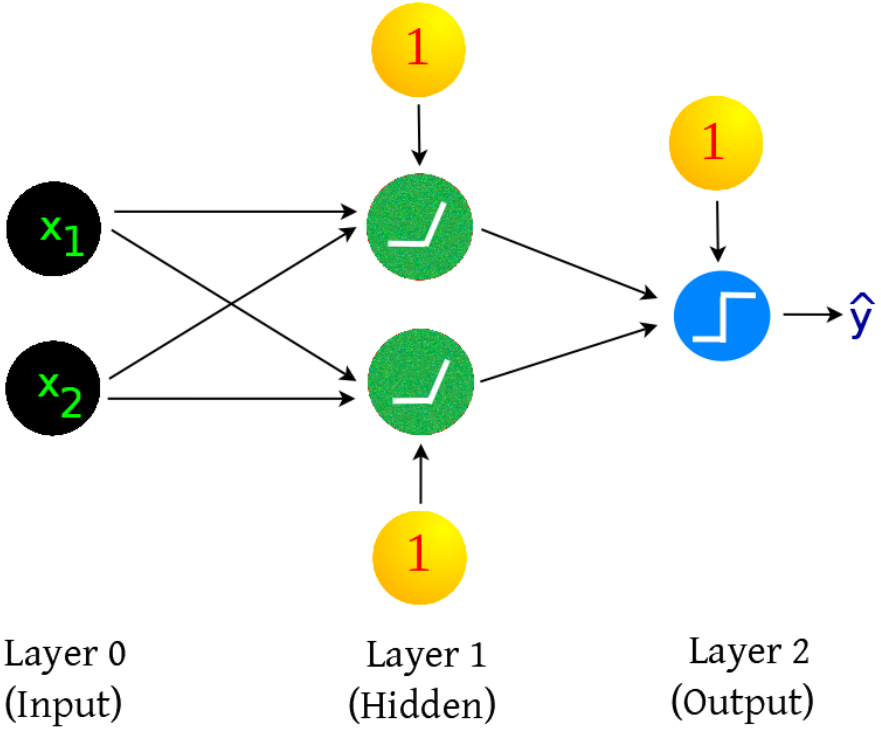
- **Idea:** The separating surface need not be linear in the ambient space and can be assumed to take some non-linear form.

Moving away from perceptron - Dealing with XOR problem

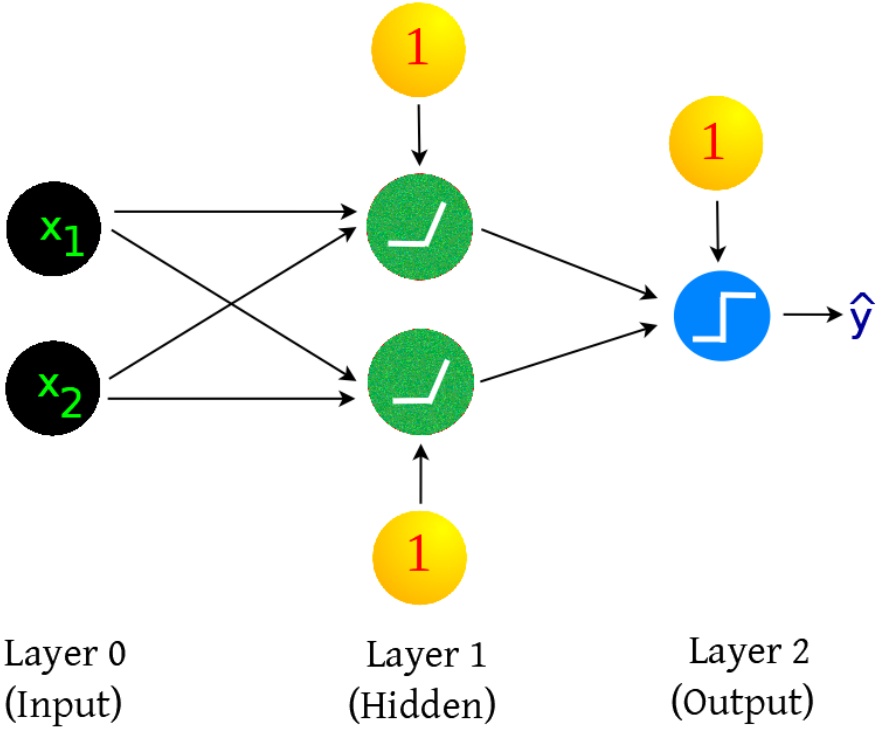


- **Idea:** The separating surface need not be linear in the ambient space and can be assumed to take some non-linear form.
- Hence for an input space $\mathcal{X}$ and output space $\mathcal{Y}$, the learned map $h : \mathcal{X} \to \mathcal{Y}$ can take some non-linear form.

Multi Layer Perceptron

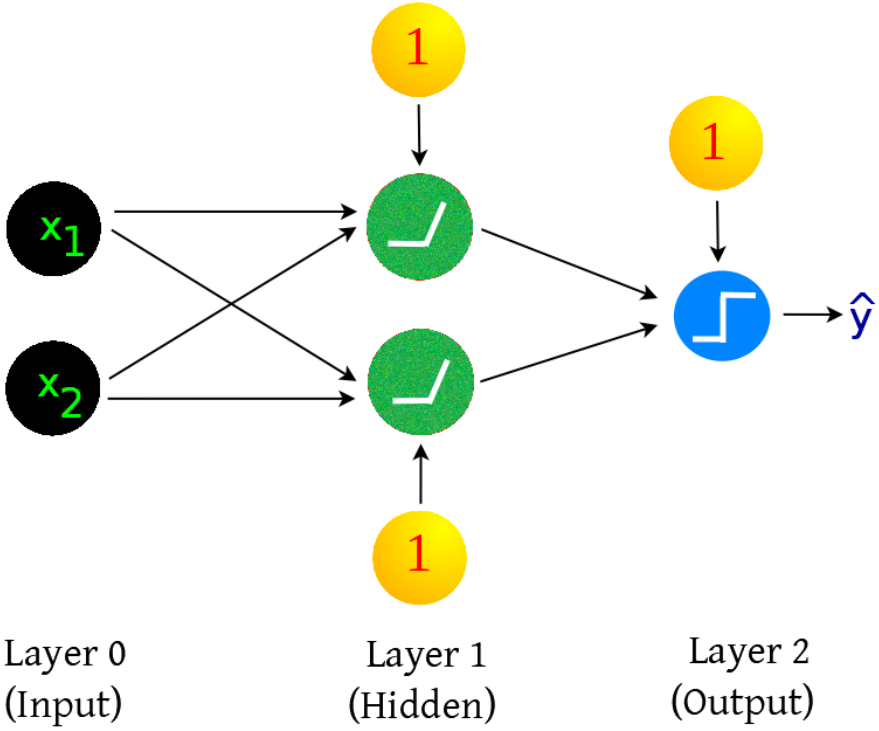


Multi Layer Perceptron



Notable features:

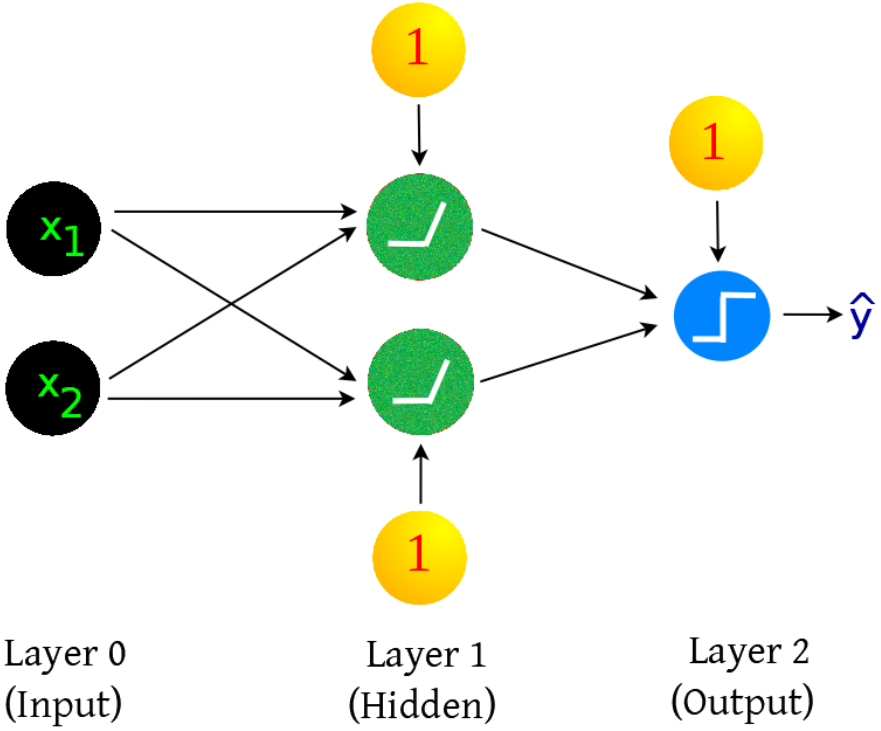
Multi Layer Perceptron



Notable features:

- Multiple layers stacked together.

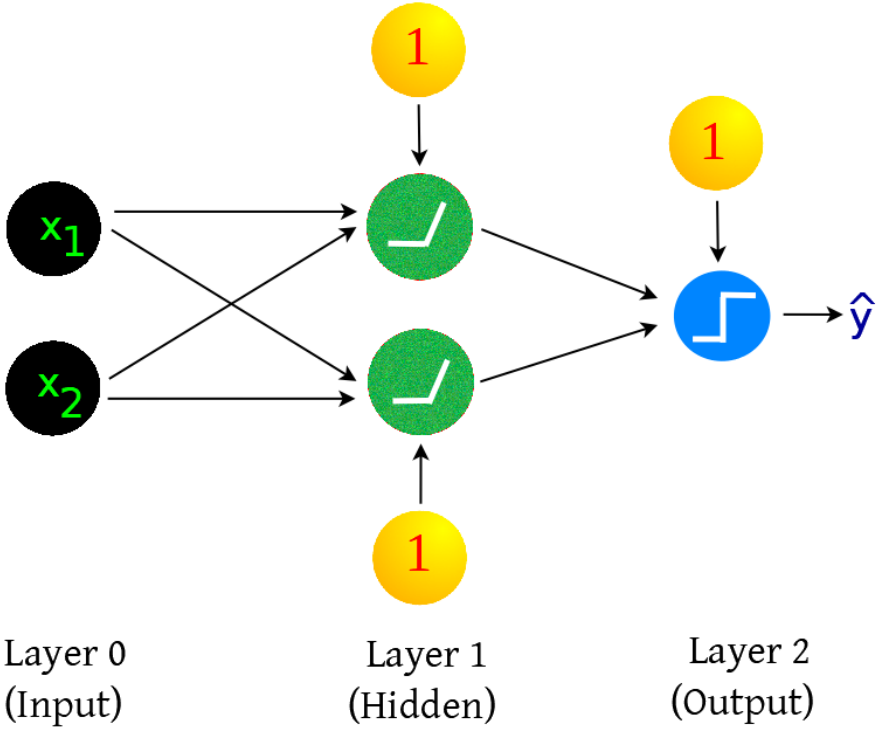
Multi Layer Perceptron



Notable features:

- Multiple layers stacked together.
- Zero-th layer usually called input layer.

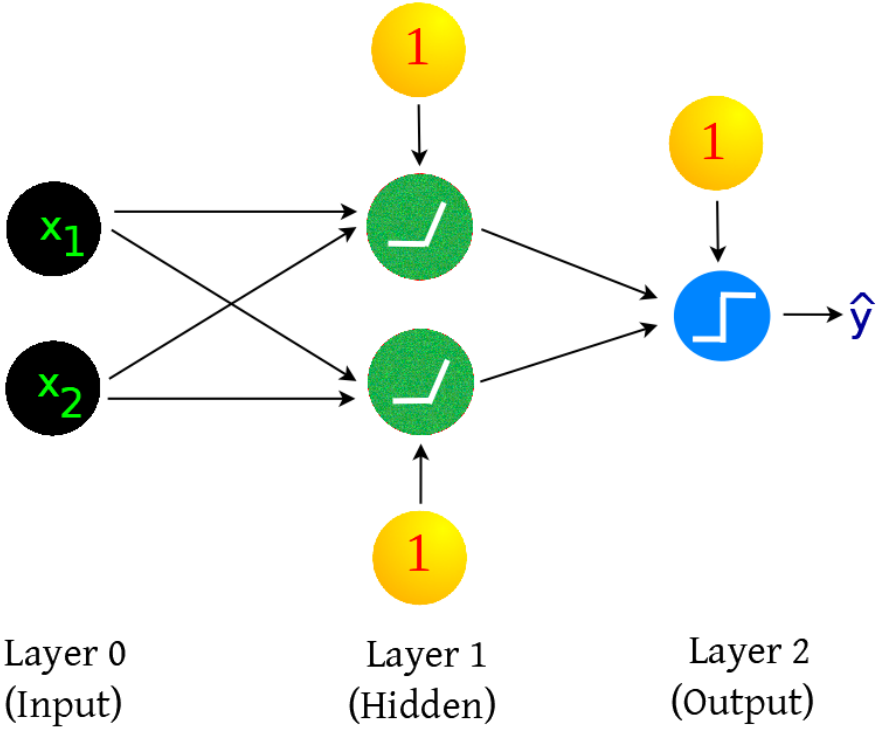
Multi Layer Perceptron



Notable features:

- Multiple layers stacked together.
- Zero-th layer usually called input layer.
- Final layer usually called output layer.

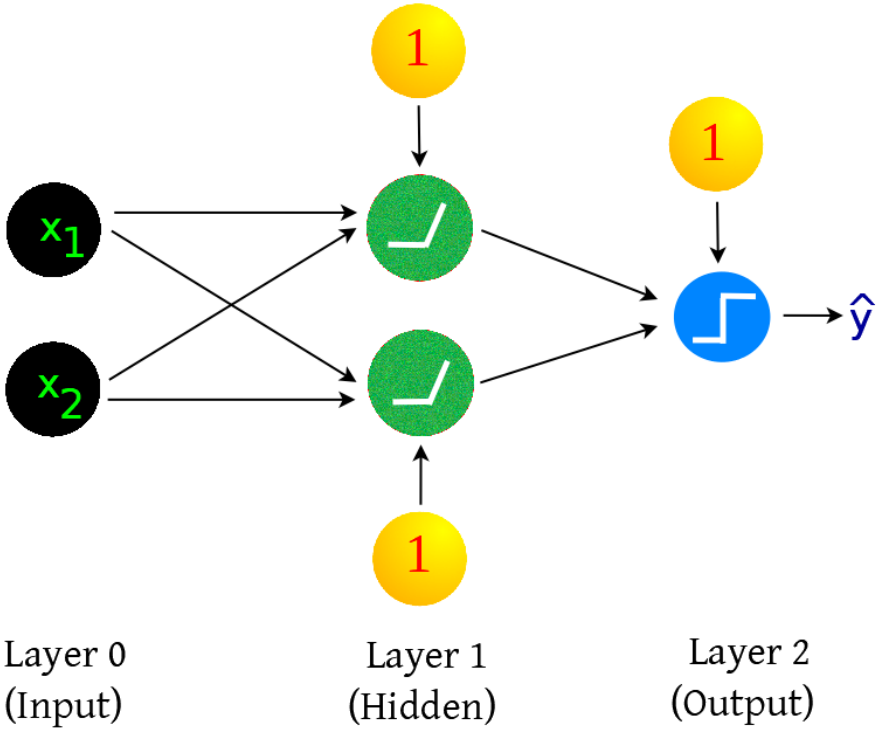
Multi Layer Perceptron



Notable features:

- Multiple layers stacked together.
- Zero-th layer usually called input layer.
- Final layer usually called output layer.
- Intermediate layers are called hidden layers.

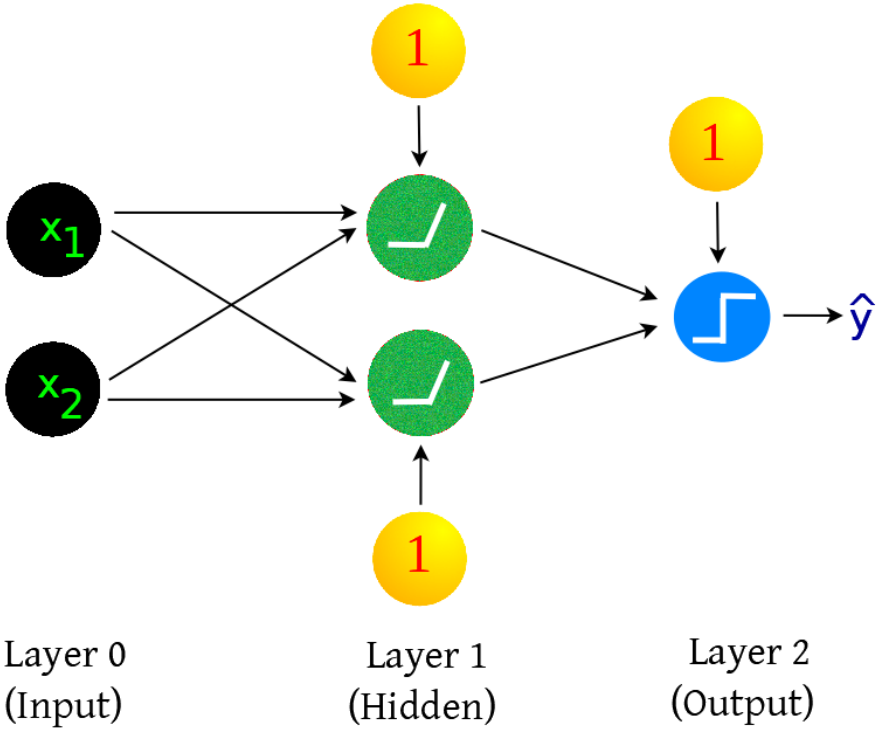
Multi Layer Perceptron



Notable features:

- Each neuron in the hidden and output layer is like a perceptron.

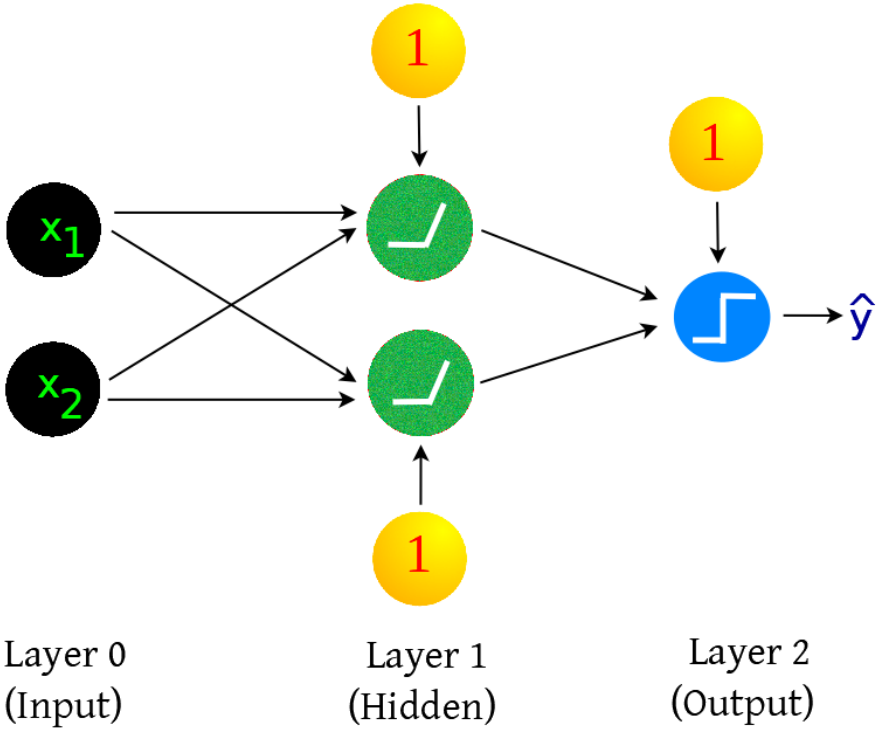
Multi Layer Perceptron



Notable features:

- Each neuron in the hidden and output layer is like a perceptron.
- However, unlike perceptron, different activation functions are used.

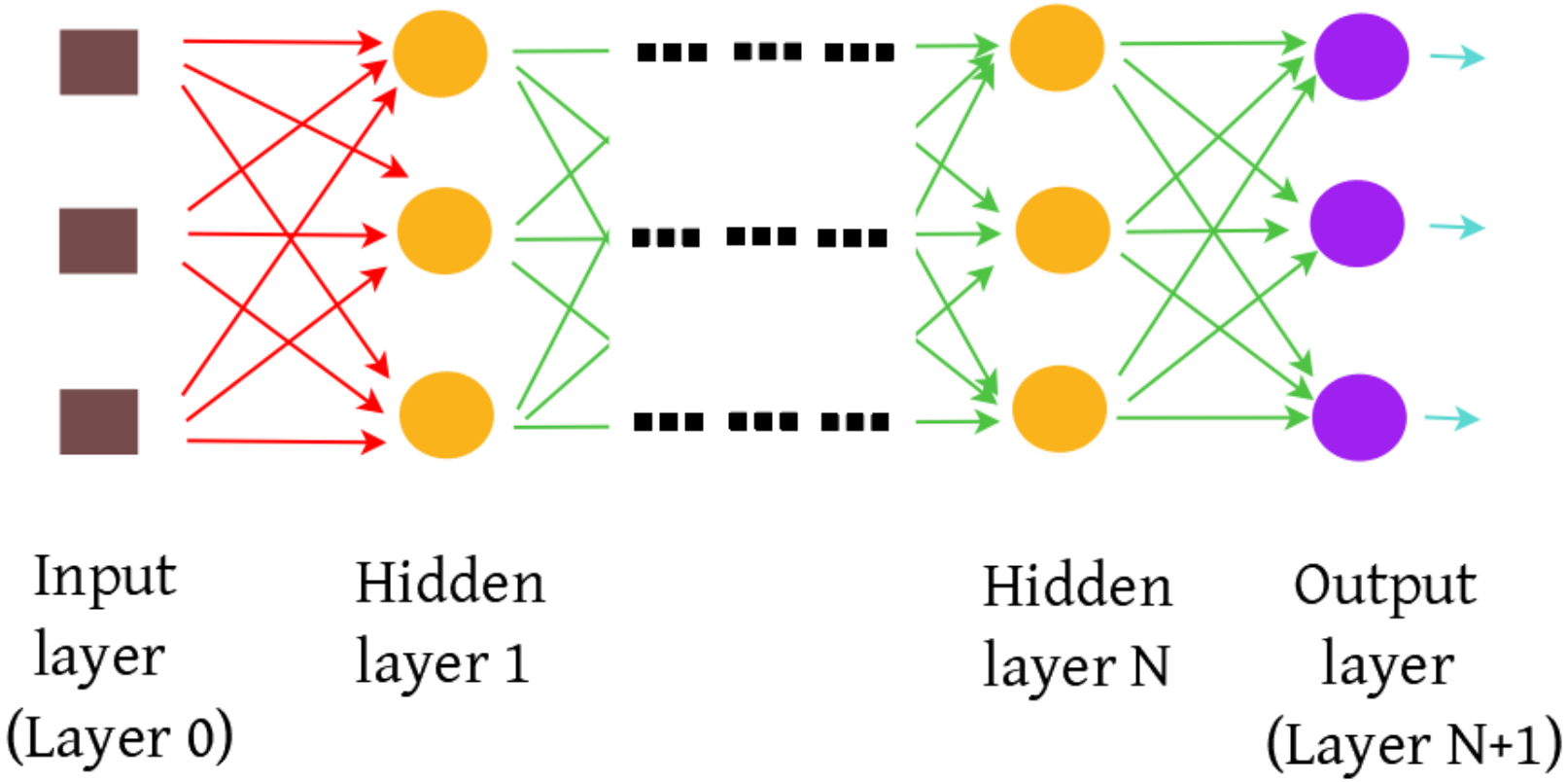
Multi Layer Perceptron



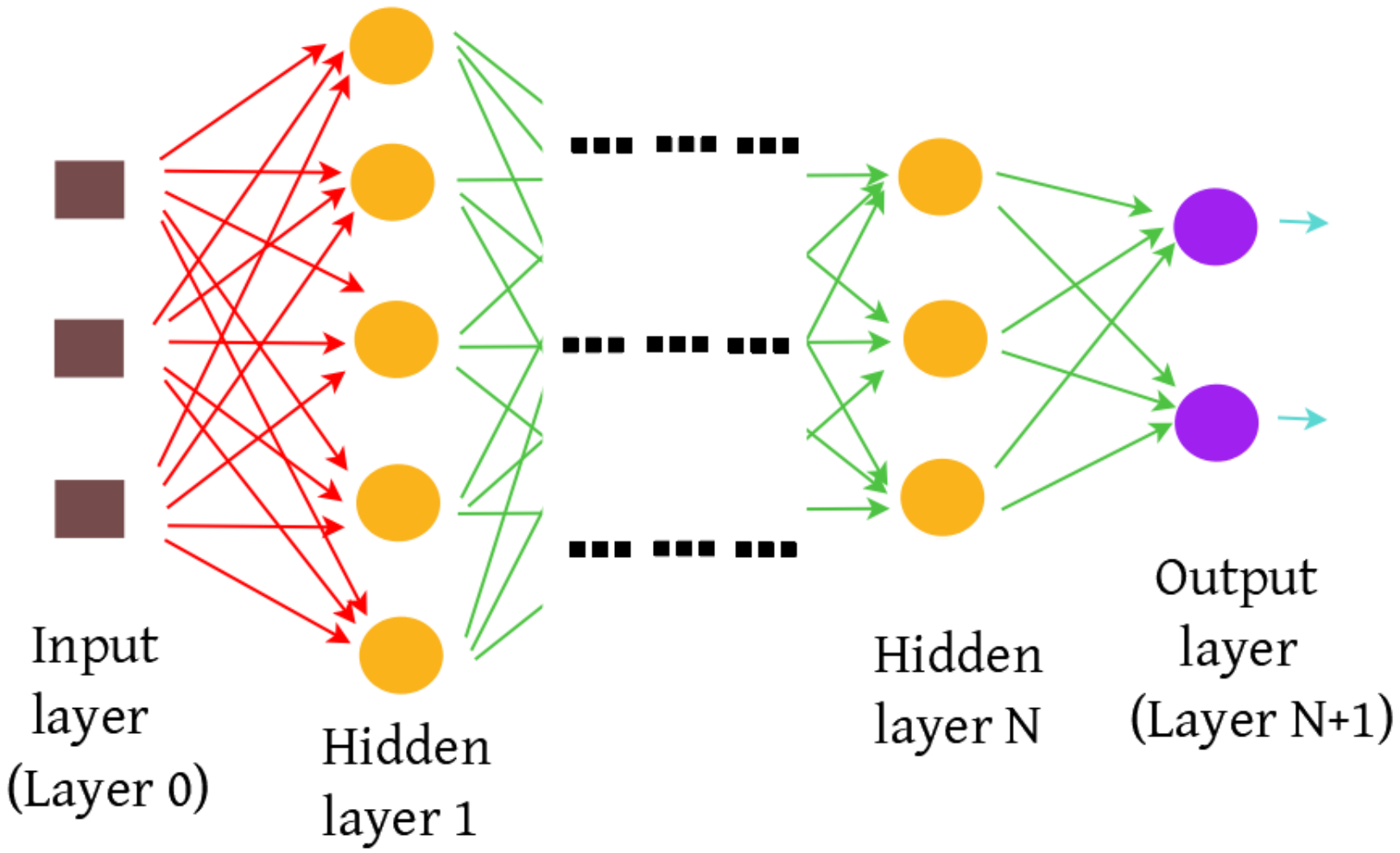
Notable features:

- Each neuron in the hidden and output layer is like a perceptron.
- However, unlike perceptron, different activation functions are used.
- $\max\{x, 0\}$ has a special name called **ReLU** (**R**ectified **L**inear **U**nit).

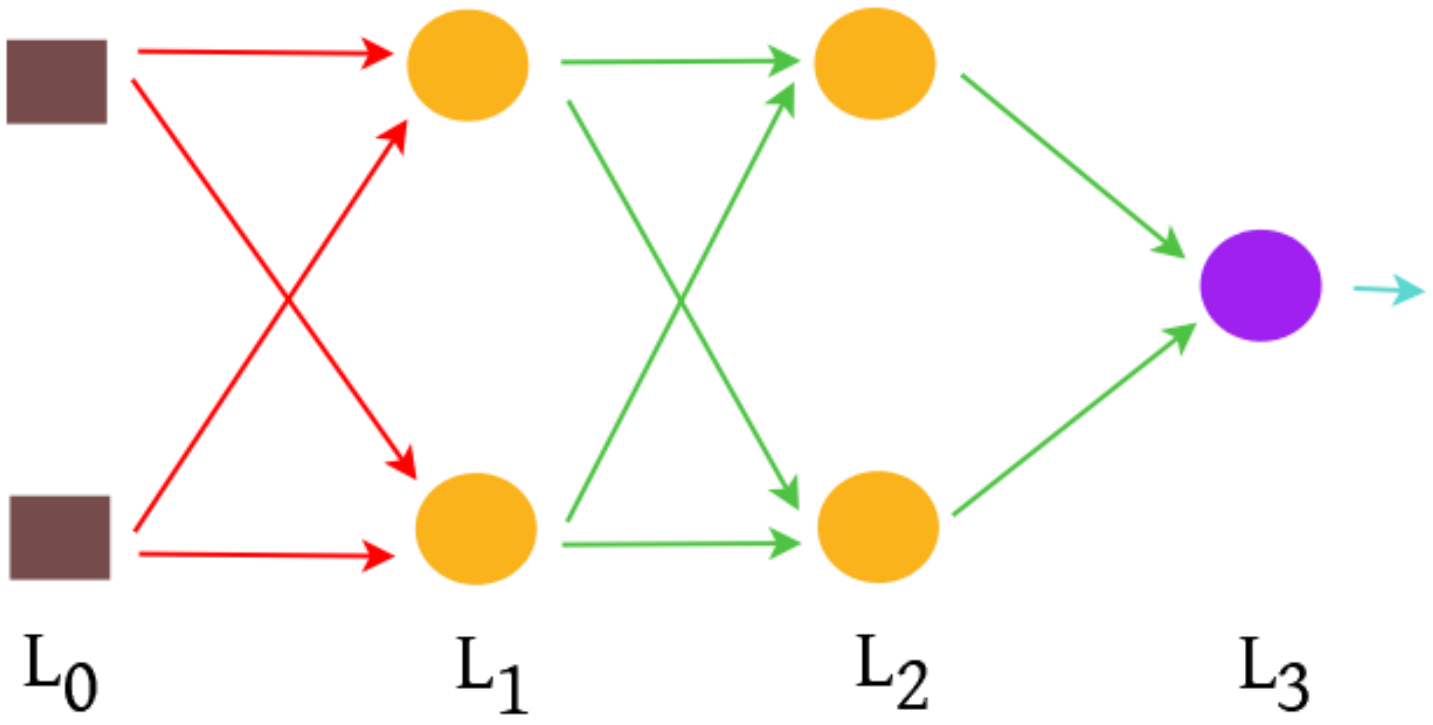
Multi Layer Perceptron



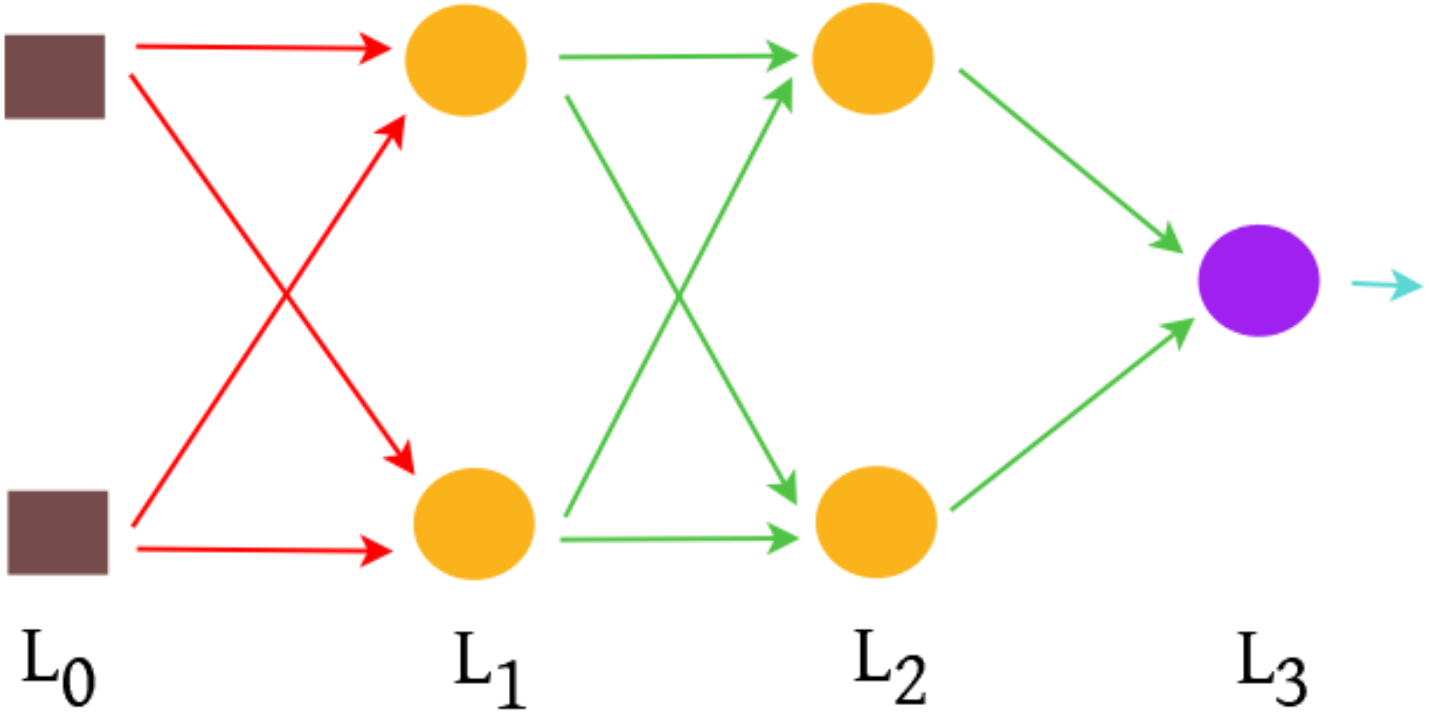
Multi Layer Perceptron



Multi Layer Perceptron - More notations

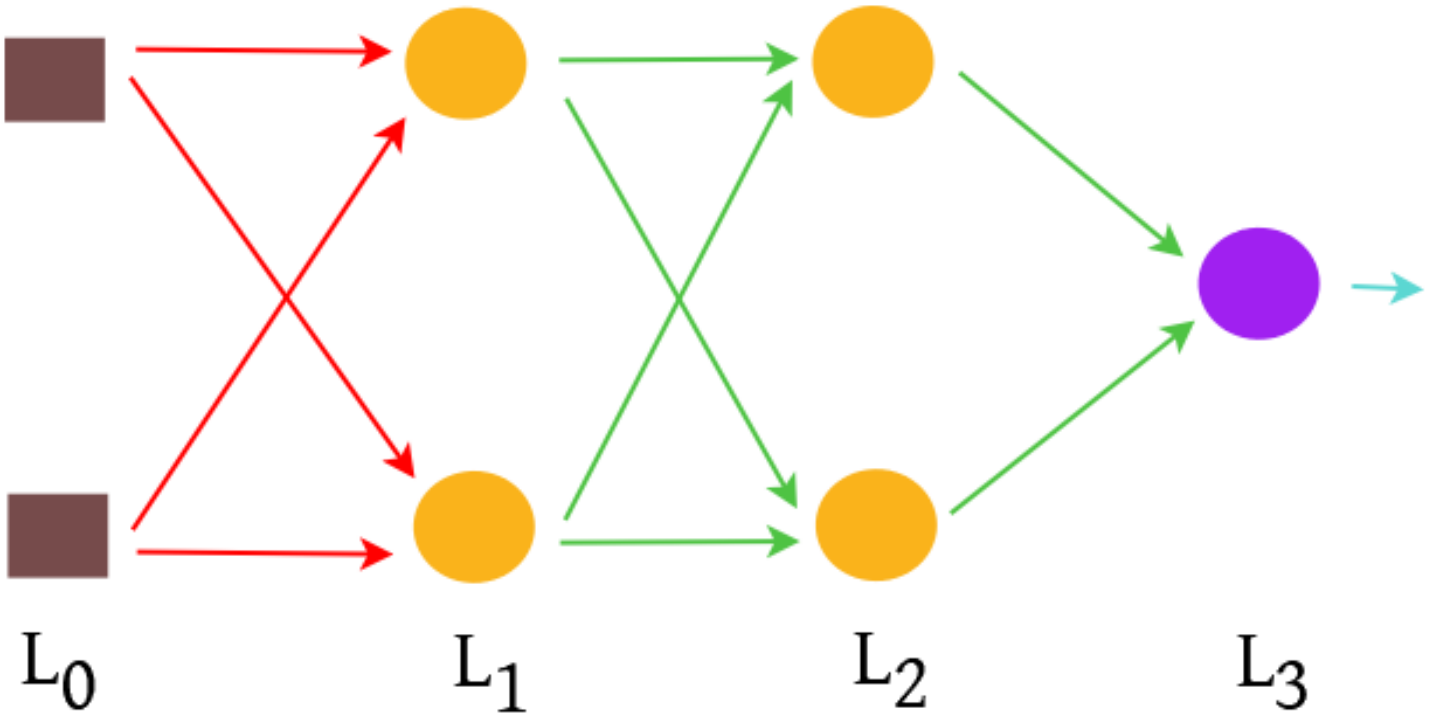


Multi Layer Perceptron - More notations



- This MLP contains an input layer L_0, 2 hidden layers denoted by L_1, L_2, and output layer L_3.

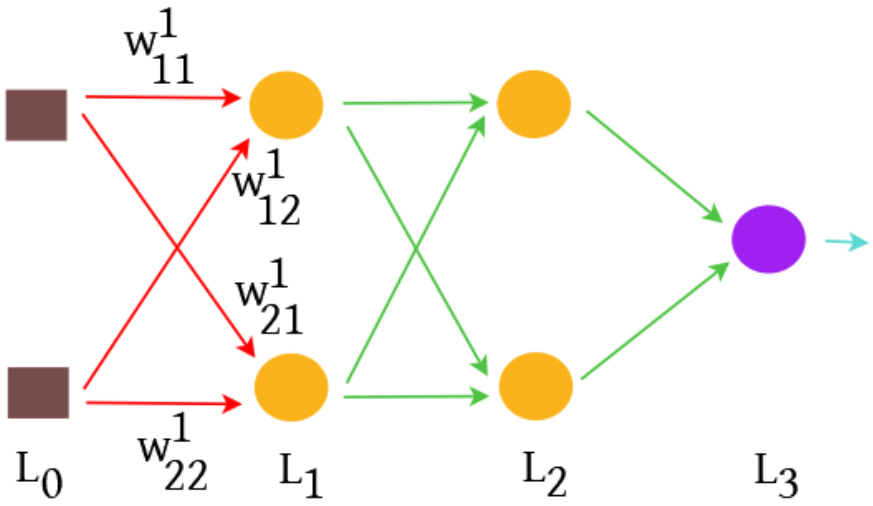
Multi Layer Perceptron - More notations



Recall:

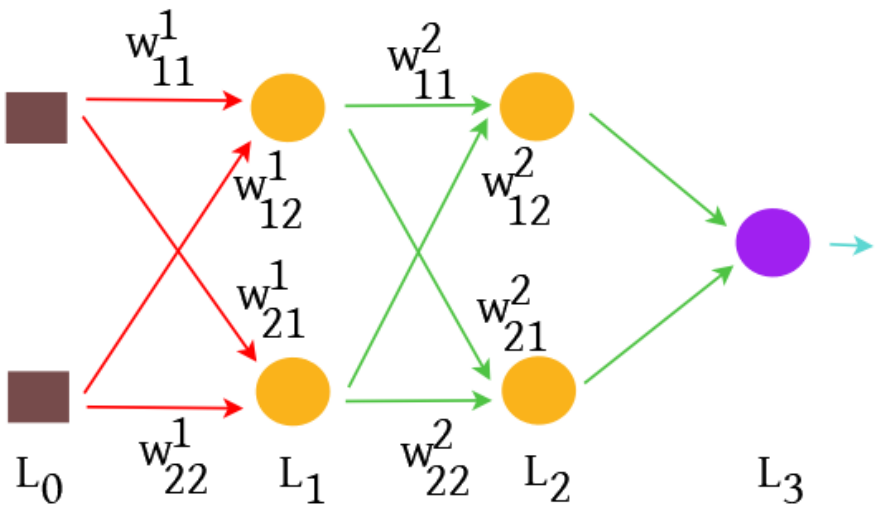
- n_k^ℓ denotes k-th neuron at ℓ-th layer.
- a_k^ℓ denotes activation of neuron n_k^ℓ.

Multi Layer Perceptron - More notations



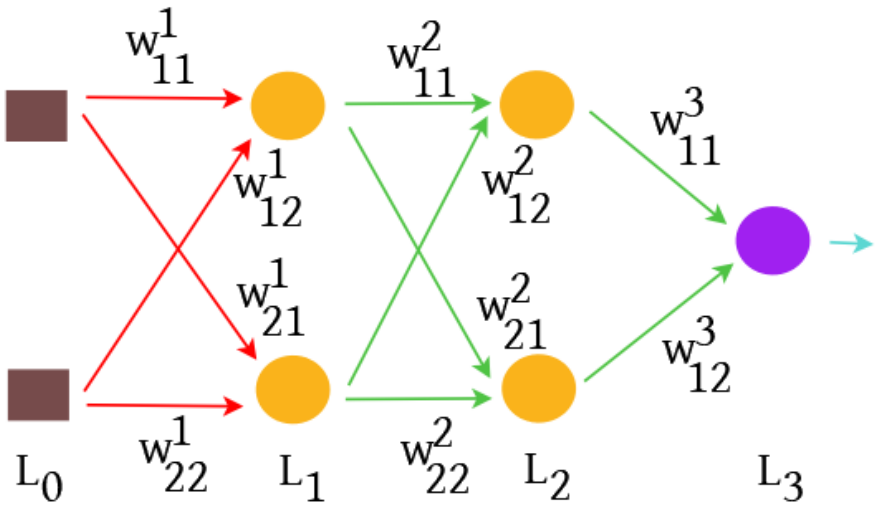
- w^{ℓ}_{ij} denotes weight of connection connecting n^{ℓ}_{i} from $n^{\ell-1}_{j}$.

Multi Layer Perceptron - More notations



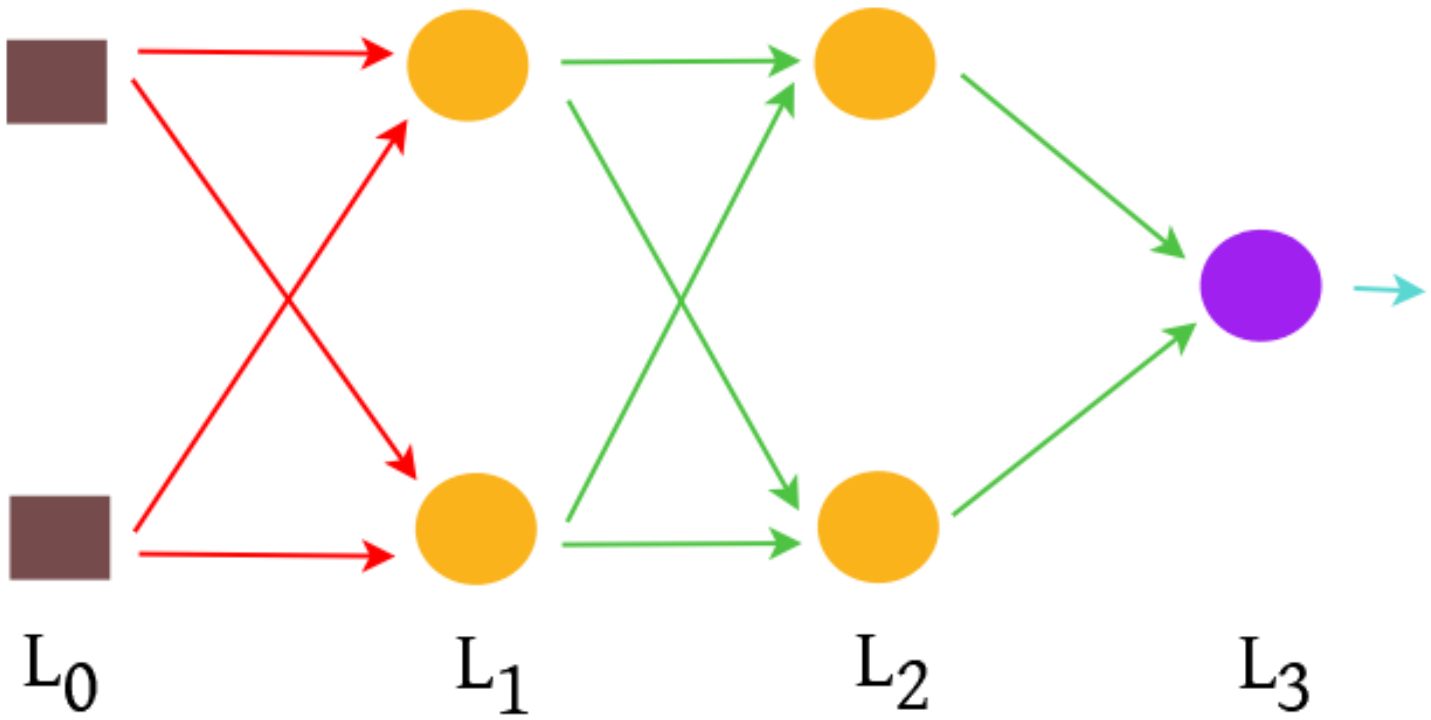
- w^{ℓ}_{ij} denotes weight of connection connecting n^{ℓ}_{i} from $n^{\ell-1}_{j}$.

Multi Layer Perceptron - More notations



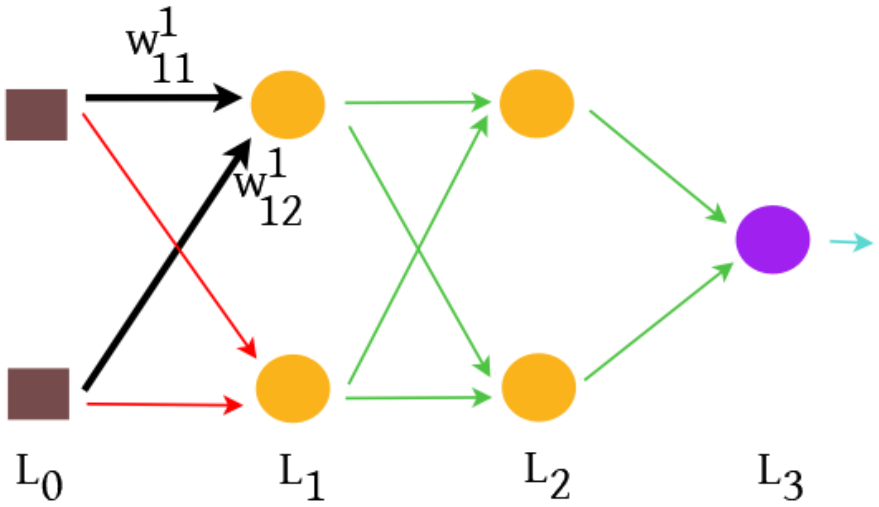
- w^{ℓ}_{ij} denotes weight of connection connecting n^{ℓ}_{i} from $n^{\ell-1}_{j}$.

Multi Layer Perceptron - More notations



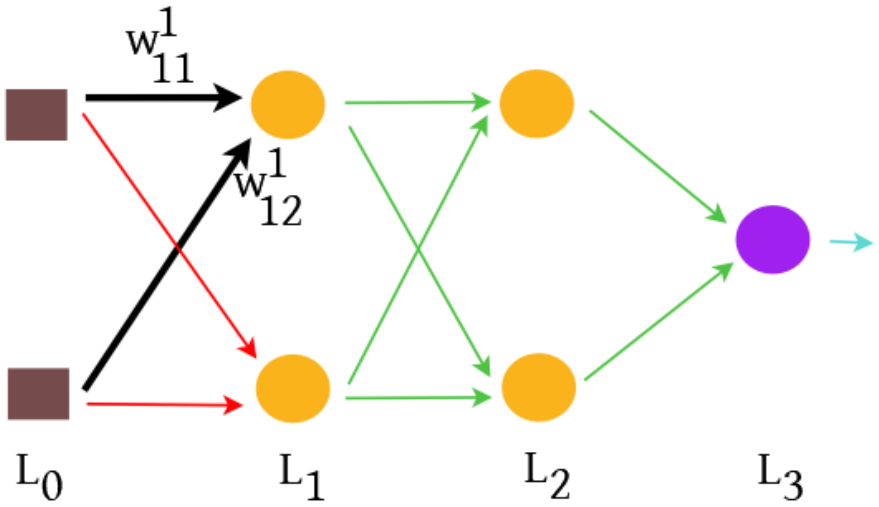
- In this particular case, the inputs are x_1 and x_2 at input layer L_0.

Multi Layer Perceptron - More notations



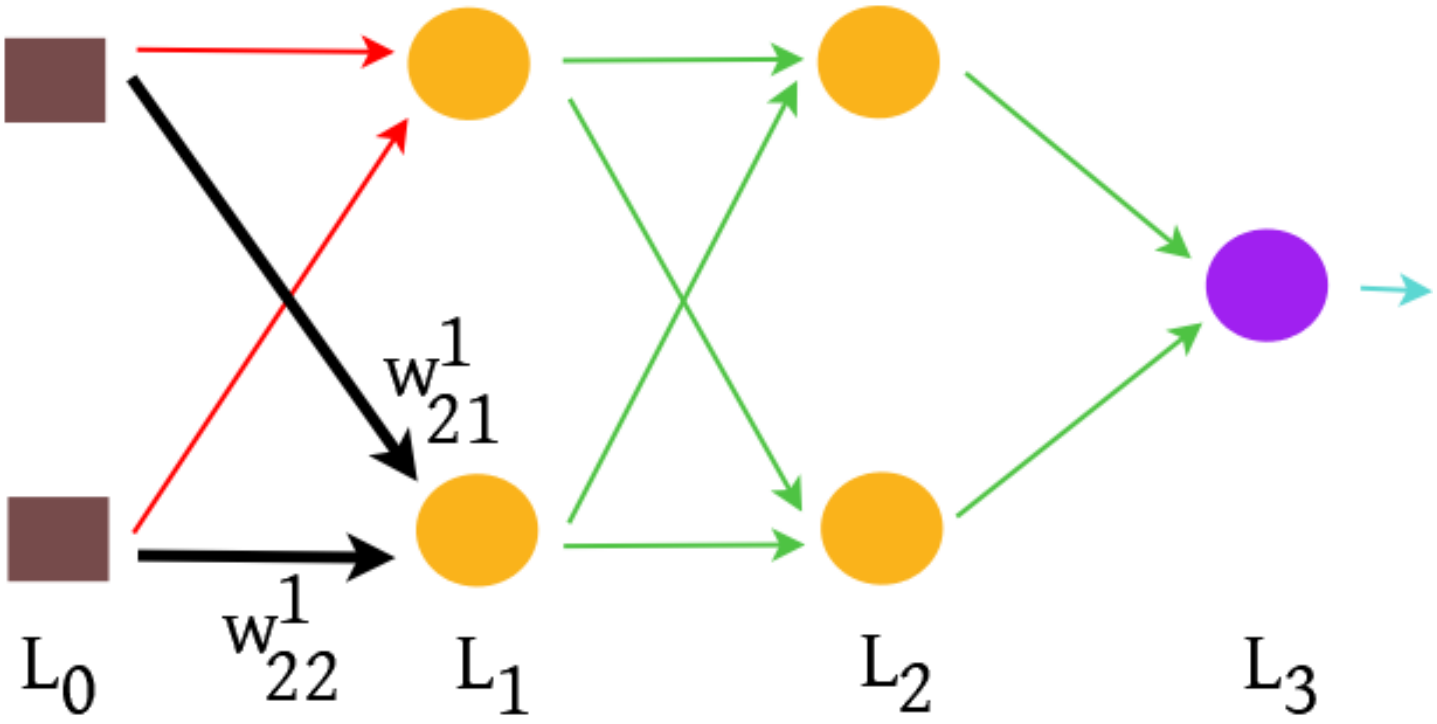
- At layer L_1:
  - At neuron n^1_1:
    - ★ $a^1_1 = \phi(w^1_{11}x_1 + w^1_{12}x_2)$.

Multi Layer Perceptron - More notations



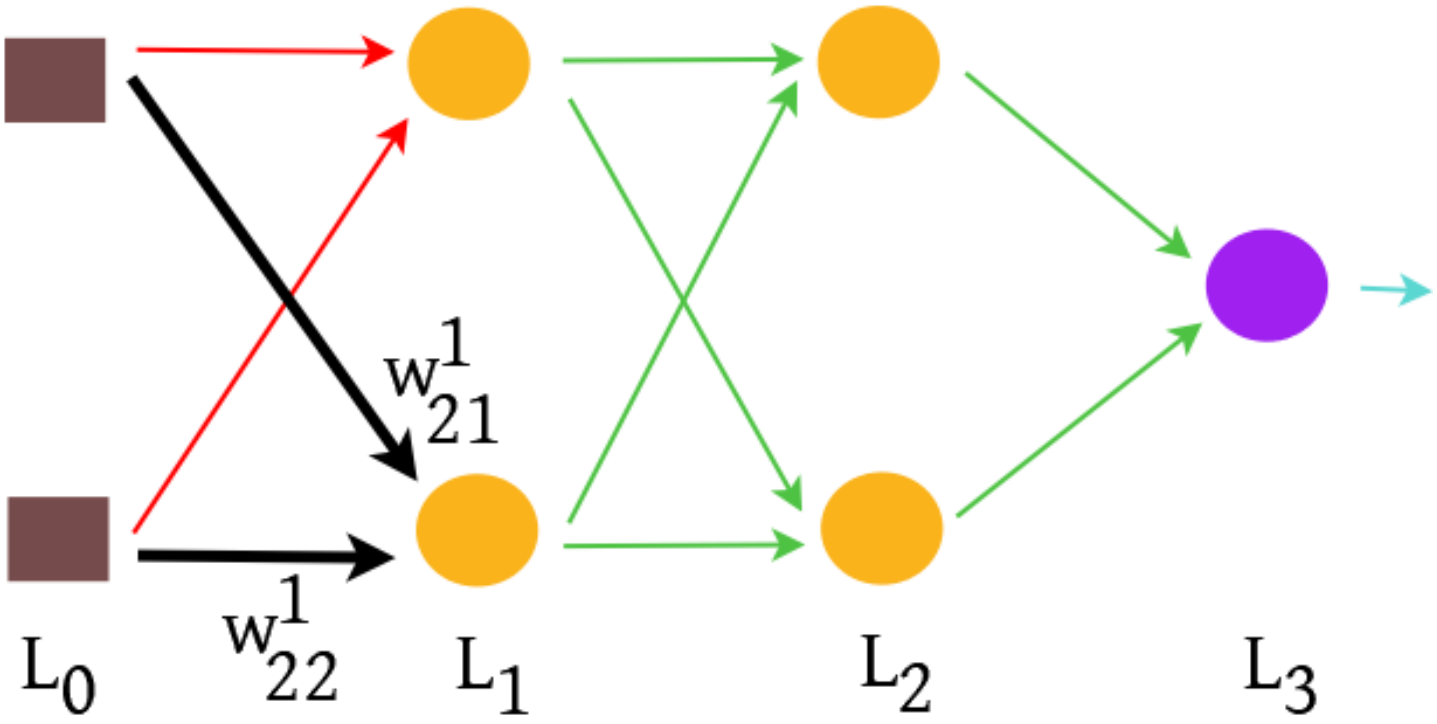
- At layer L_1:
  - At neuron n^1_1:
    - ★ $a^1_1 = \phi(w^1_{11}x_1 + w^1_{12}x_2) =: \phi(z^1_1)$.

Multi Layer Perceptron - More notations



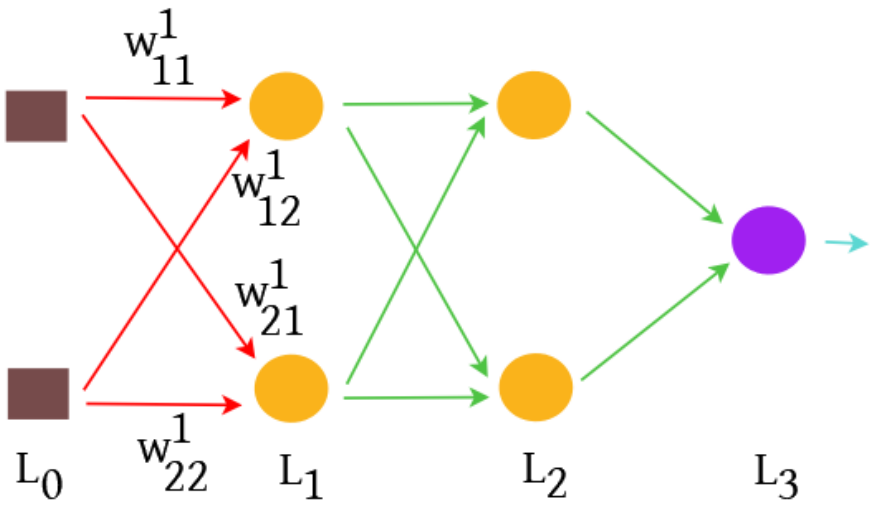
- At layer L_1:
  - At neuron n^1_2:
    - $a^1_2 = \phi(w^1_{21}x_1 + w^1_{22}x_2)$.

Multi Layer Perceptron - More notations



- At layer L_1:
  - At neuron n^1_2:
    - ★ $a^1_2 = \phi(w^1_{21}x_1 + w^1_{22}x_2) =: \phi(z^1_2)$.

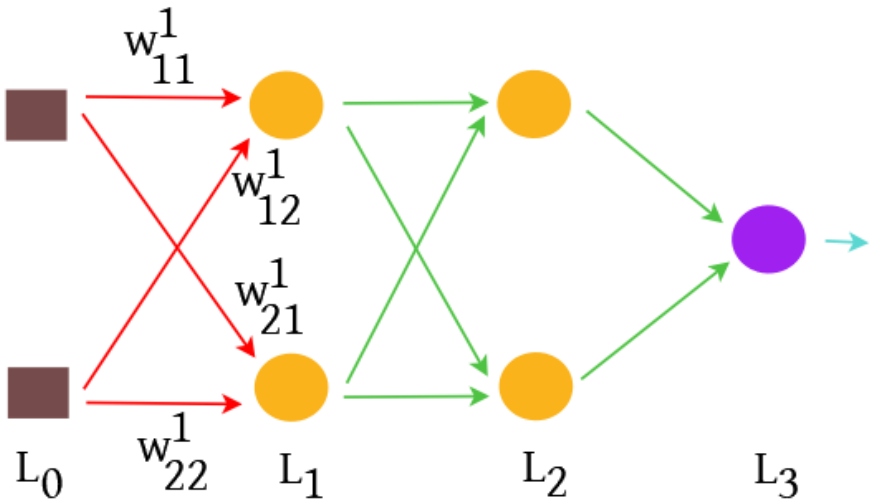
Multi Layer Perceptron - More notations



- At layer L_1:

$$\begin{bmatrix} a_1^1 \\ a_2^1 \end{bmatrix} = \begin{bmatrix} \phi(z_1^1) \\ \phi(z_2^1) \end{bmatrix} = \begin{bmatrix} \phi(w_{11}^1 x_1 + w_{12}^1 x_2) \\ \phi(w_{21}^1 x_1 + w_{22}^1 x_2) \end{bmatrix}$$

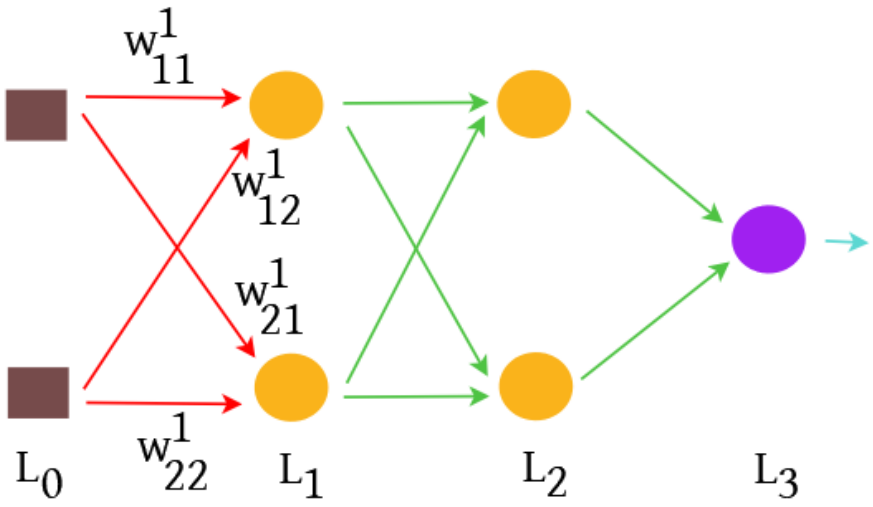
Multi Layer Perceptron - More notations



- Letting $W^1 = \begin{bmatrix} w^1_{11} & w^1_{12} \\ w^1_{21} & w^1_{22} \end{bmatrix}$ and $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, we have at layer L_1:

$$\begin{bmatrix} a^1_1 \\ a^1_2 \end{bmatrix} = \phi\left(\begin{bmatrix} z^1_1 \\ z^1_2 \end{bmatrix}\right) = \phi\left(\begin{bmatrix} w^1_{11}x_1 + w^1_{12}x_2 \\ w^1_{21}x_1 + w^1_{22}x_2 \end{bmatrix}\right) = \phi(W^1 x)$$

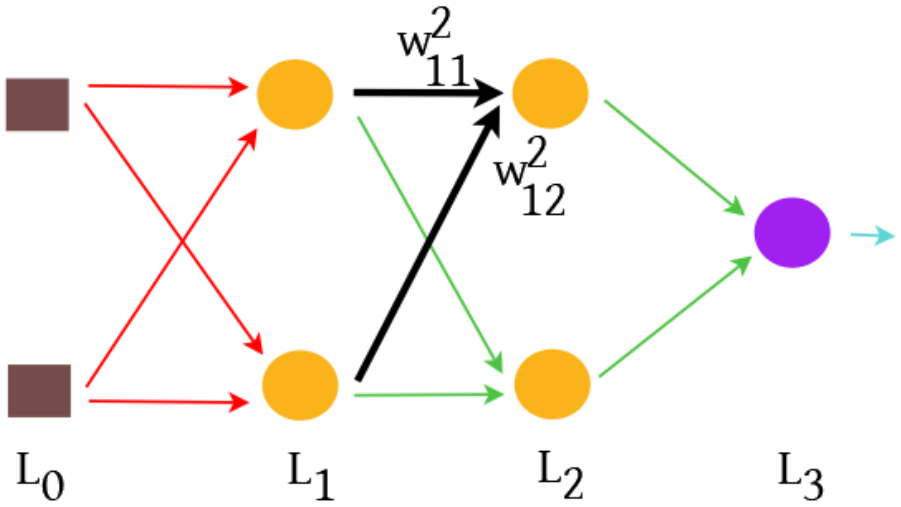
Multi Layer Perceptron - More notations



- Letting $a^1 = \begin{bmatrix} a^1_1 \\ a^1_2 \end{bmatrix}$, we have at layer L_1:

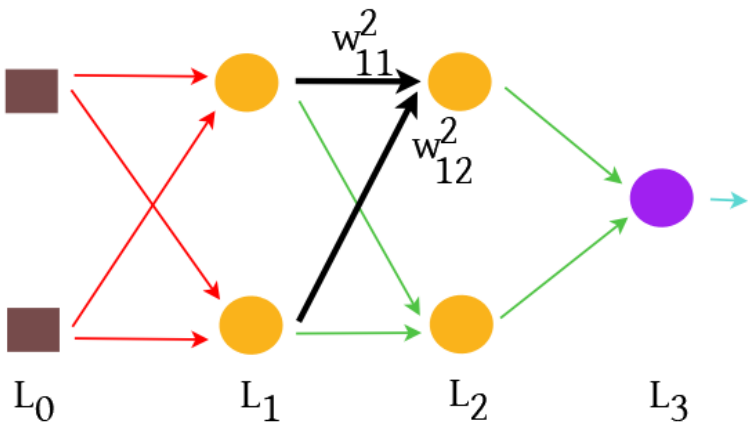
$$a^1 = \begin{bmatrix} a^1_1 \\ a^1_2 \end{bmatrix} = \phi(W^1 x)$$

Multi Layer Perceptron - More notations



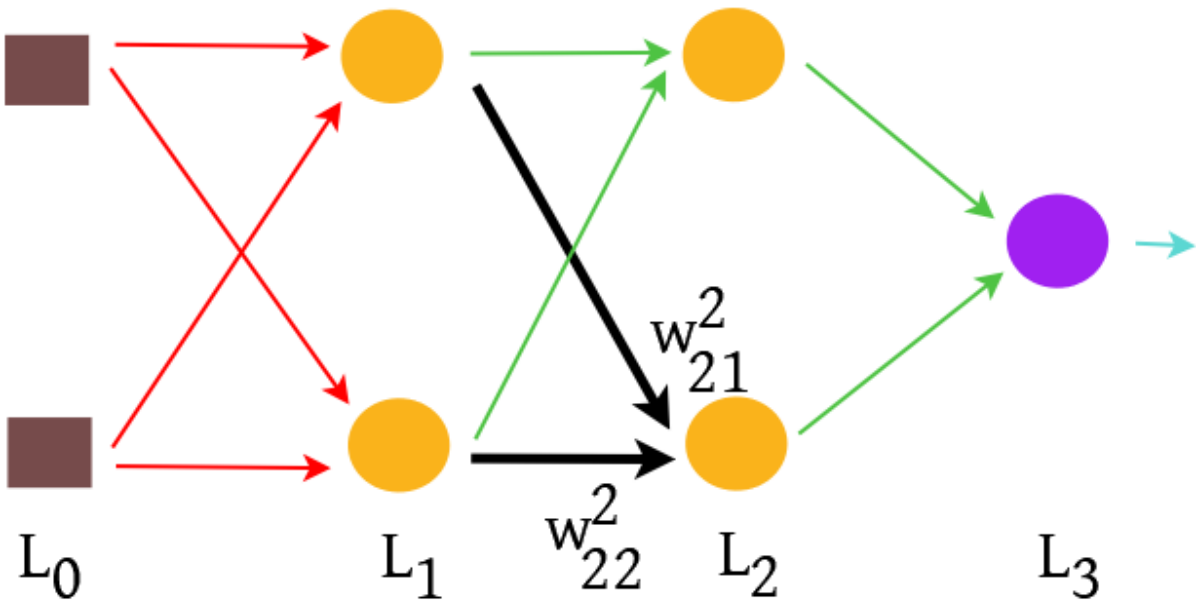
- At layer L_2:
  - At neuron n^2_1:
    - ★ $a^2_1 = \phi(w^2_{11} a^1_1 + w^2_{12} a^1_2)$.

Multi Layer Perceptron - More notations



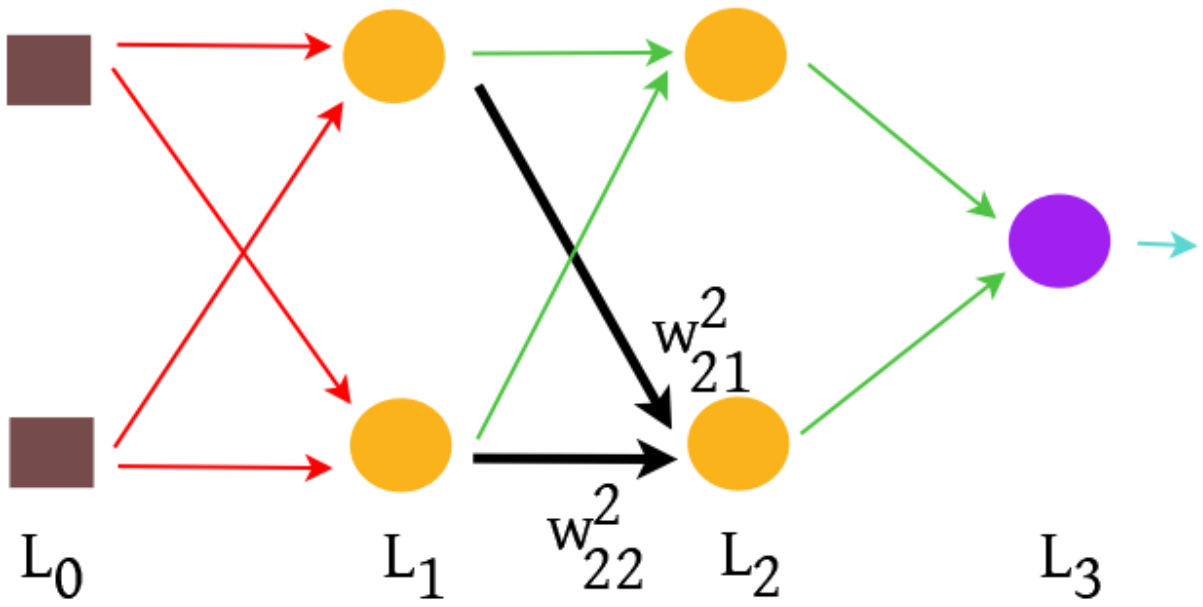
- At layer L_2:
  - At neuron n^2_1:
    - ★ $a^2_1 = \phi(w^2_{11}a^1_1 + w^2_{12}a^1_2) =: \phi(z^2_1)$.

Multi Layer Perceptron - More notations



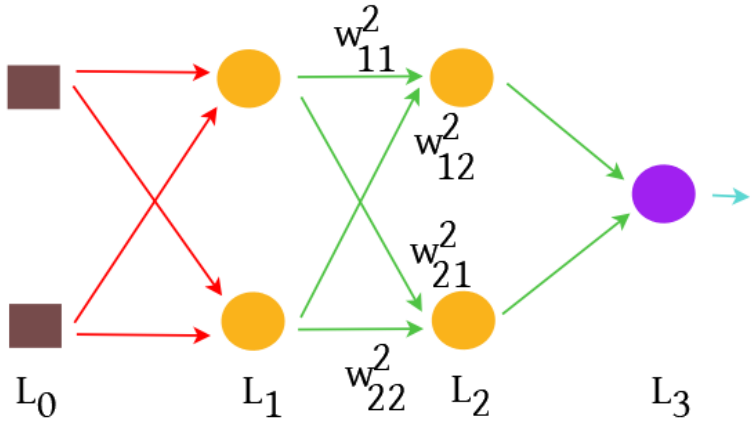
- At layer L_2:
  - At neuron n_2^2:
    - ★ $a_2^2 = \phi(w_{21}^2 a_1^1 + w_{22}^2 a_2^1)$.

Multi Layer Perceptron - More notations



- At layer L_2:
  - At neuron n_2^2:
    - ★ $a_2^2 = \phi(w_{21}^2 a_1^1 + w_{22}^2 a_2^1) =: \phi(z_2^2)$.

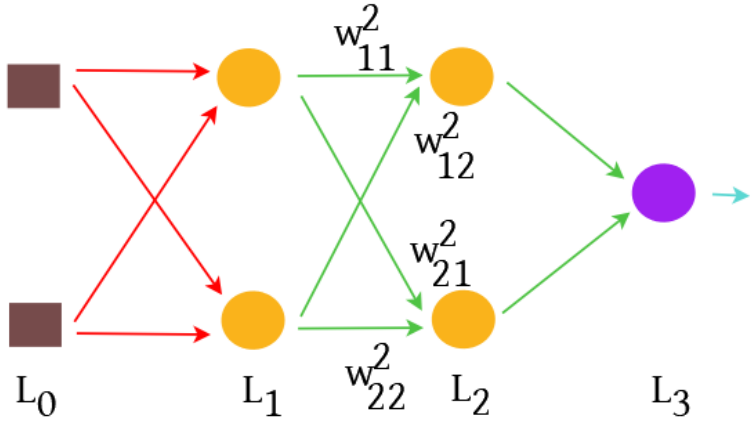
Multi Layer Perceptron - More notations



- At layer L_2:

$$a^2 = \begin{bmatrix} a_1^2 \\ a_2^2 \end{bmatrix} = \begin{bmatrix} \phi(z_1^2) \\ \phi(z_2^2) \end{bmatrix} = \begin{bmatrix} \phi(w_{11}^2 a_1^1 + w_{12}^2 a_2^1) \\ \phi(w_{21}^2 a_1^1 + w_{22}^2 a_2^1) \end{bmatrix}$$

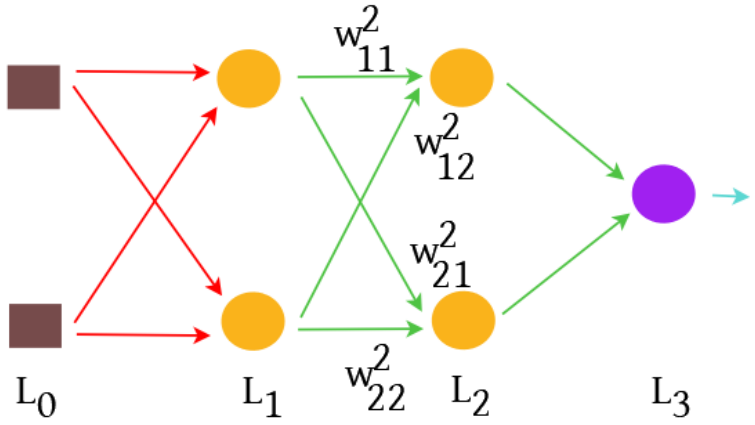
Multi Layer Perceptron - More notations



- Letting $W^2 = \begin{bmatrix} w^2_{11} & w^2_{12} \\ w^2_{21} & w^2_{22} \end{bmatrix}$, we have at layer L_2:

$$a^2 = \begin{bmatrix} a^2_1 \\ a^2_2 \end{bmatrix} = \phi\left(\begin{bmatrix} z^2_1 \\ z^2_2 \end{bmatrix}\right) = \phi\left(\begin{bmatrix} w^2_{11}a^1_1 + w^2_{12}a^1_2 \\ w^2_{21}a^1_1 + w^2_{22}a^1_2 \end{bmatrix}\right) = \phi\left(W^2 \begin{bmatrix} a^1_1 \\ a^1_2 \end{bmatrix}\right)$$

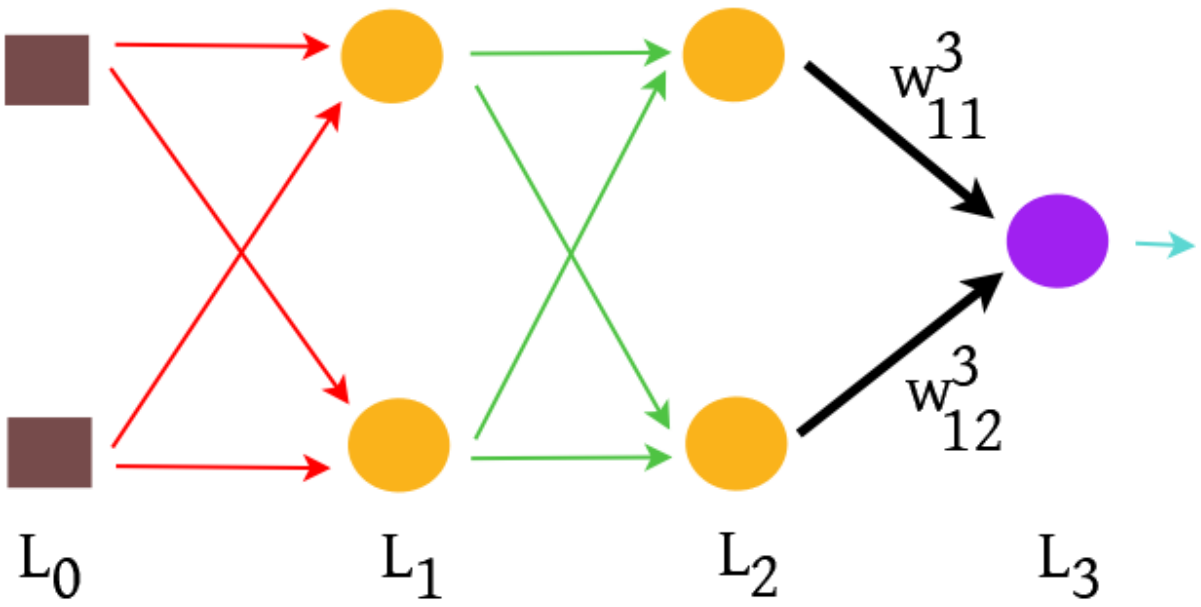
Multi Layer Perceptron - More notations



- We have at layer L_2:

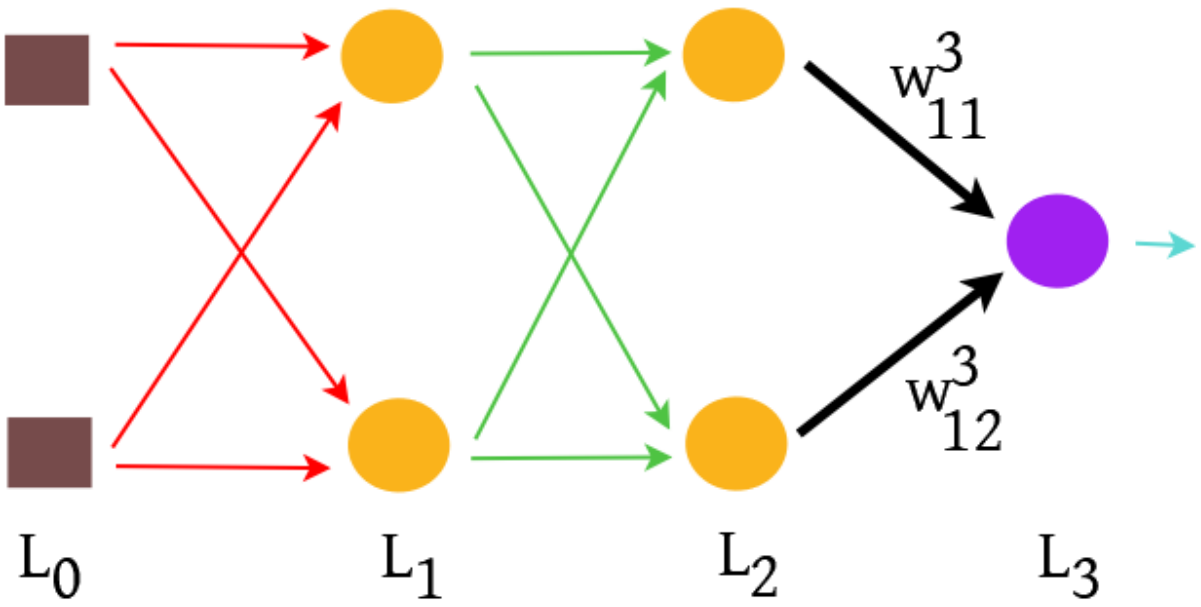
$$a^2 = \begin{bmatrix} a^2_1 \\ a^2_2 \end{bmatrix} = \phi\left(\begin{bmatrix} z^2_1 \\ z^2_2 \end{bmatrix}\right) = \phi\left(W^2 \begin{bmatrix} a^1_1 \\ a^1_2 \end{bmatrix}\right) = \phi(W^2 a^1)$$

Multi Layer Perceptron - More notations



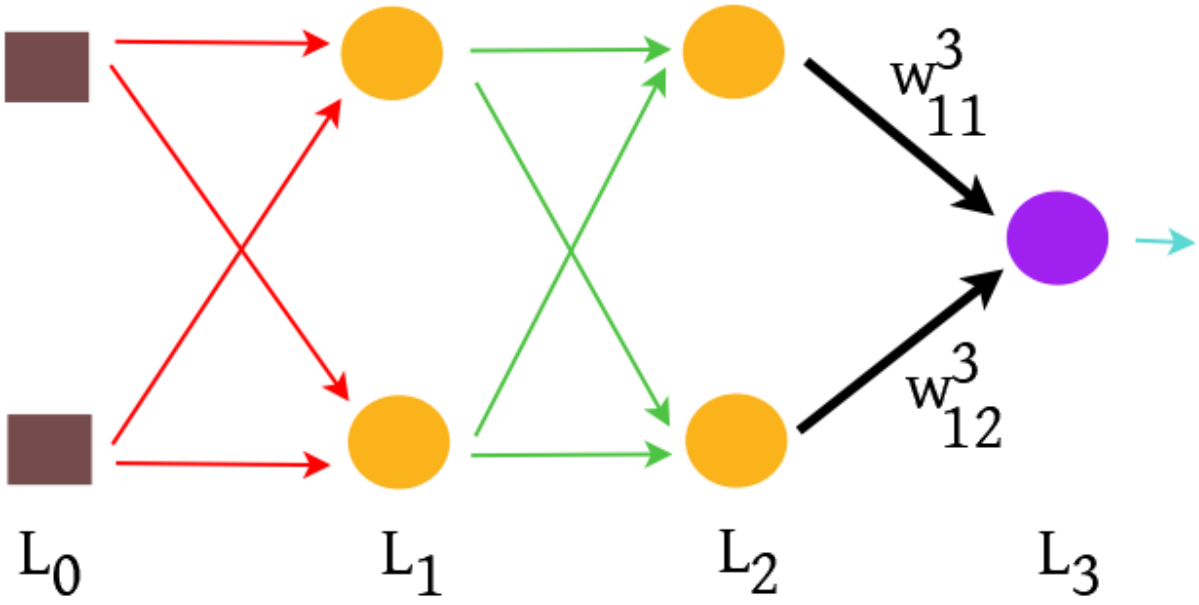
- At layer L_3:
  - At neuron n^3_1:
    - ★ $a^3_1 = \phi(w^3_{11} a^2_1 + w^3_{12} a^2_2)$.

Multi Layer Perceptron - More notations



- At layer L_3:
  - At neuron n^3_1:
    - ★ $a^3_1 = \phi(w^3_{11}a^2_1 + w^3_{12}a^2_2) =: \phi(z^3_1)$.

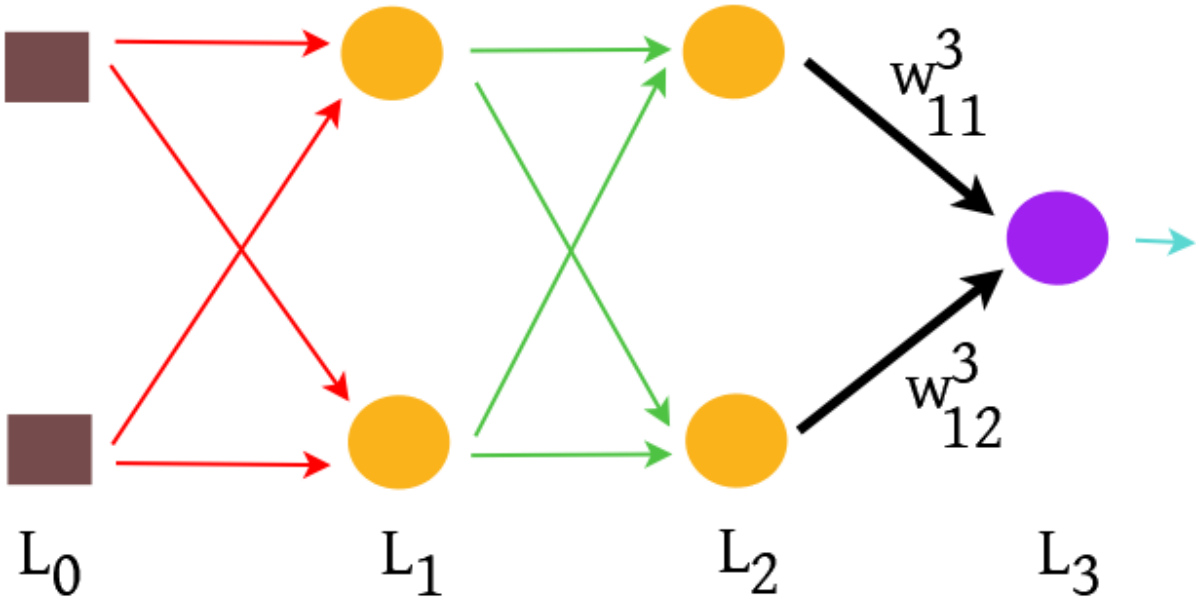
Multi Layer Perceptron - More notations



- At layer L_3:

$$a^3 = [a^3_1] = [\phi(z^3_1)] = [\phi(w^3_{11}a^2_1 + w^3_{12}a^2_2)]$$

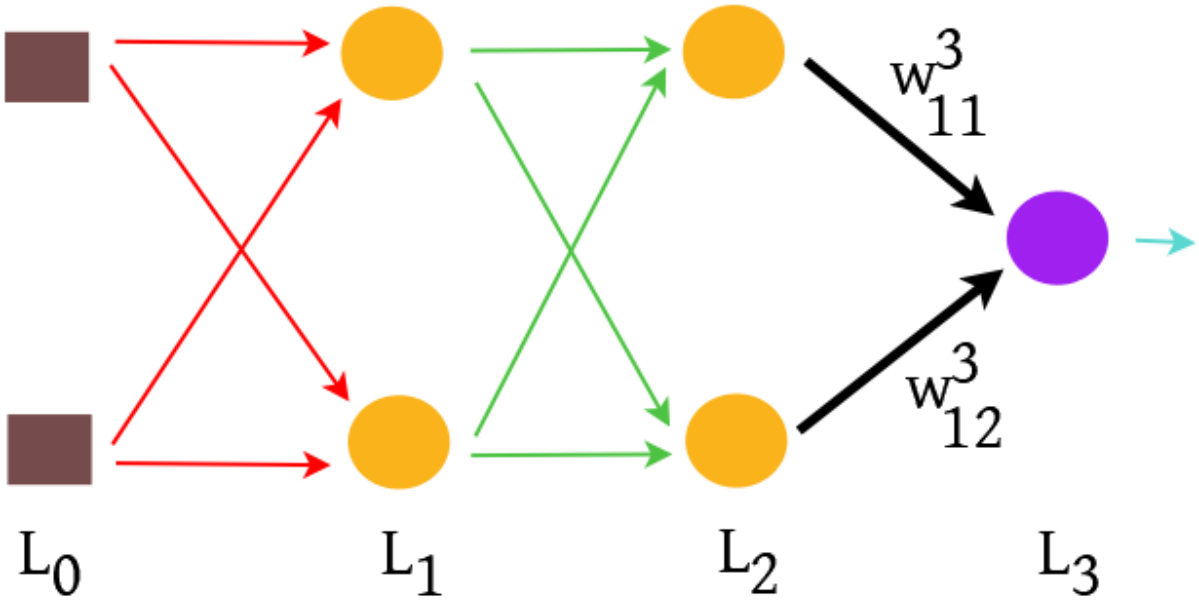
Multi Layer Perceptron - More notations



- Letting $W^3 = \begin{bmatrix} w^3_{11} & w^3_{12} \end{bmatrix}$, we have at layer L_3:

$$a^3 = \left[a^3_1\right] = \phi\left(\left[z^3_1\right]\right) = \phi\left(\left[w^3_{11}a^2_1 + w^3_{12}a^2_2\right]\right) = \phi\left(W^3 \begin{bmatrix} a^2_1 \\ a^2_2 \end{bmatrix}\right)$$

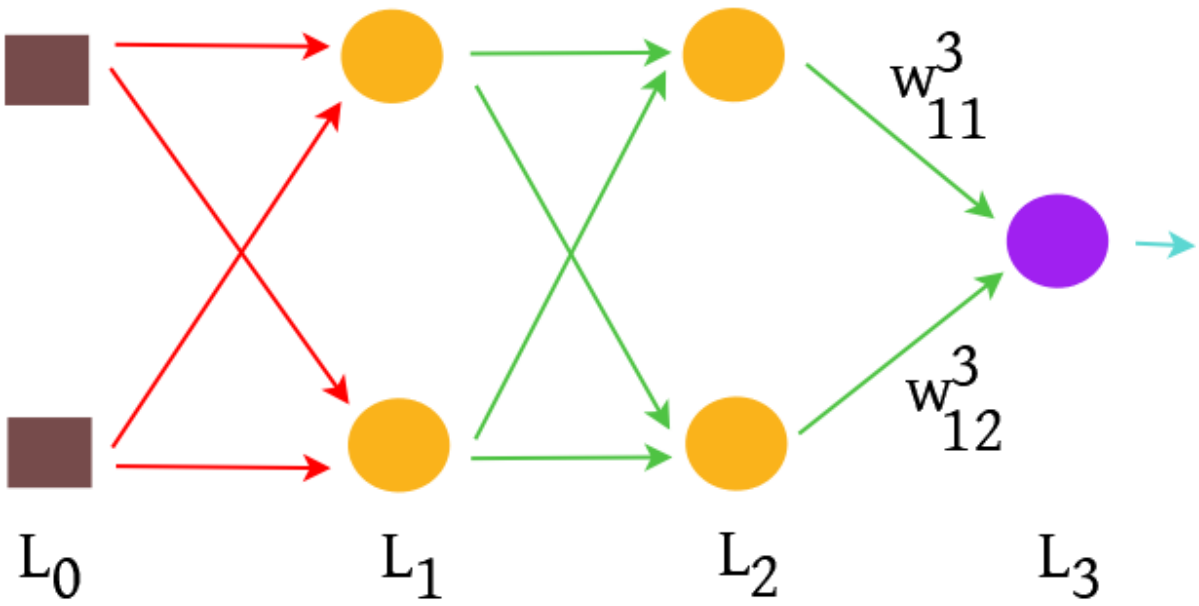
Multi Layer Perceptron - More notations



- Letting $W^3 = \begin{bmatrix} w^3_{11} & w^3_{12} \end{bmatrix}$, we have at layer L_3:

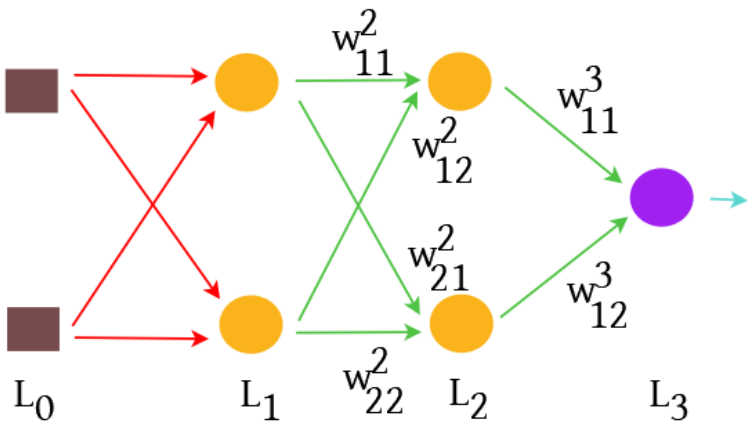
$$a^3 = \left[a^3_1\right] = \phi\left(\left[z^3_1\right]\right) = \phi\left(W^3 \begin{bmatrix} a^2_1 \\ a^2_2 \end{bmatrix}\right) = \phi(W^3 a^2)$$

Multi Layer Perceptron - More notations



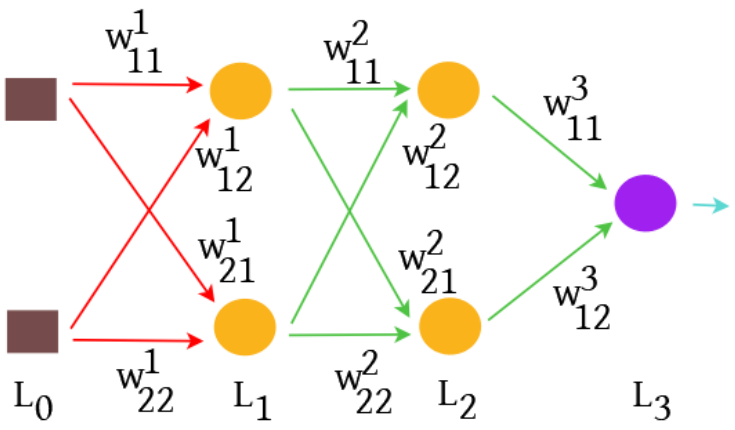
$$a^3 = \phi(W^3 a^2)$$

Multi Layer Perceptron - More notations



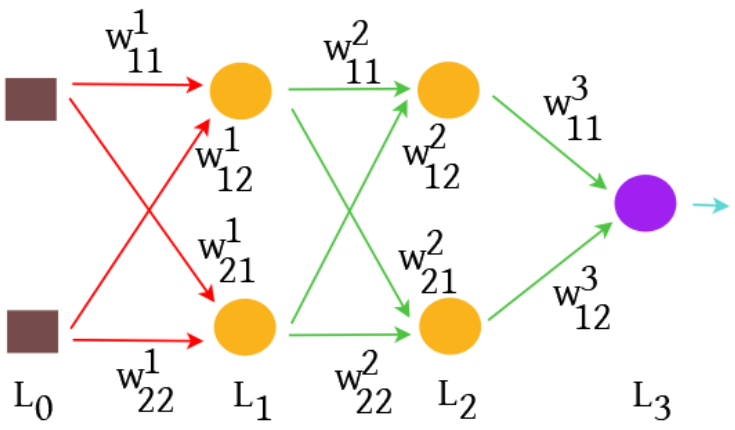
$$a^3 = \phi(W^3 a^2) = \phi(W^3 \phi(W^2 a^1))$$

Multi Layer Perceptron - More notations



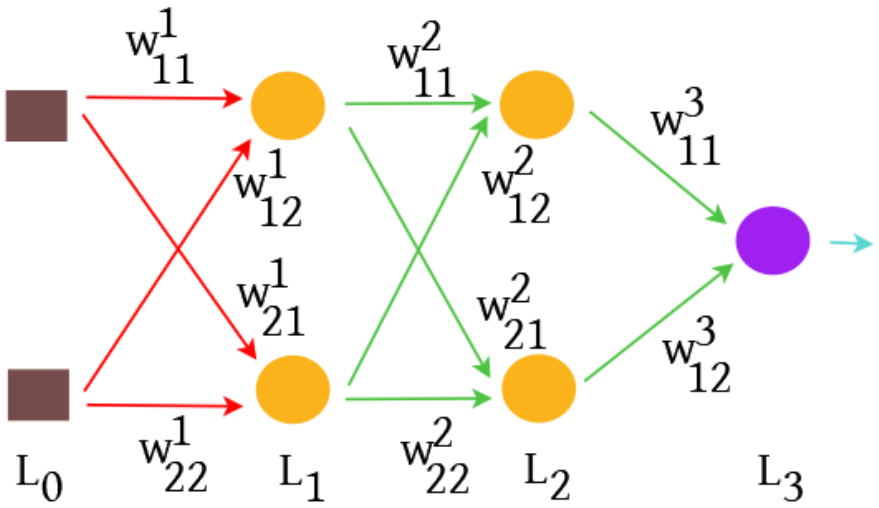
$$a^3 = \phi(W^3 a^2) = \phi(W^3 \phi(W^2 a^1)) = \phi(W^3 \phi(W^2 \phi(W^1 x)))$$

Multi Layer Perceptron - More notations



$$\hat{y} = a^3 = \phi(W^3 a^2) = \phi(W^3 \phi(W^2 a^1)) = \phi(W^3 \phi(W^2 \phi(W^1 x)))$$

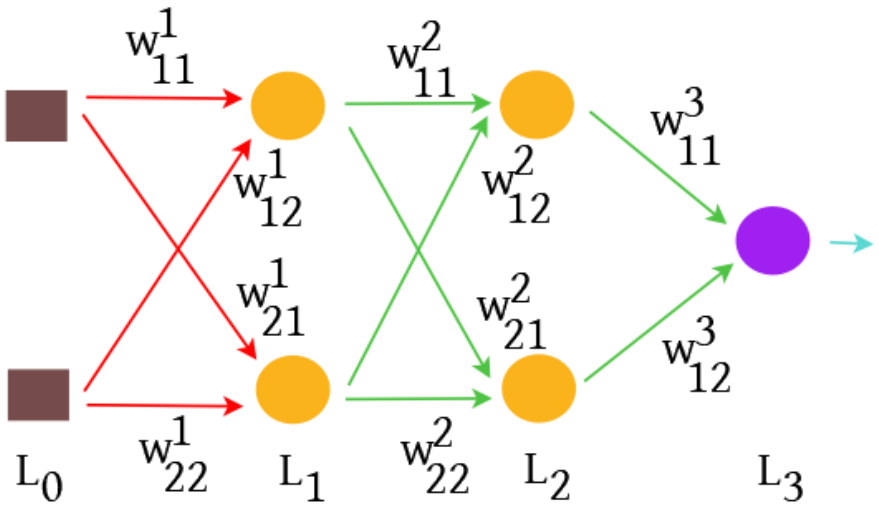
Multi Layer Perceptron - Data Perspective



Given data (x, y), multi layer perceptron predicts:

$$\hat{y} = \phi(W^3\phi(W^2\phi(W^1x)))$$

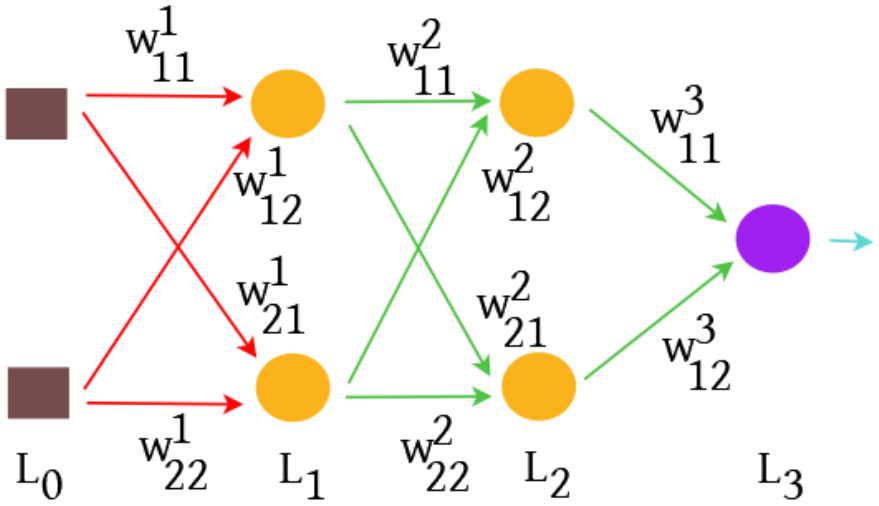
Multi Layer Perceptron - Data Perspective



Given data (x, y), multi layer perceptron predicts:

$$\hat{y} = \phi(W^3\phi(W^2\phi(W^1x))) =: \text{MLP}(x)$$

Multi Layer Perceptron - Data Perspective



Given data (x, y), multi layer perceptron predicts:

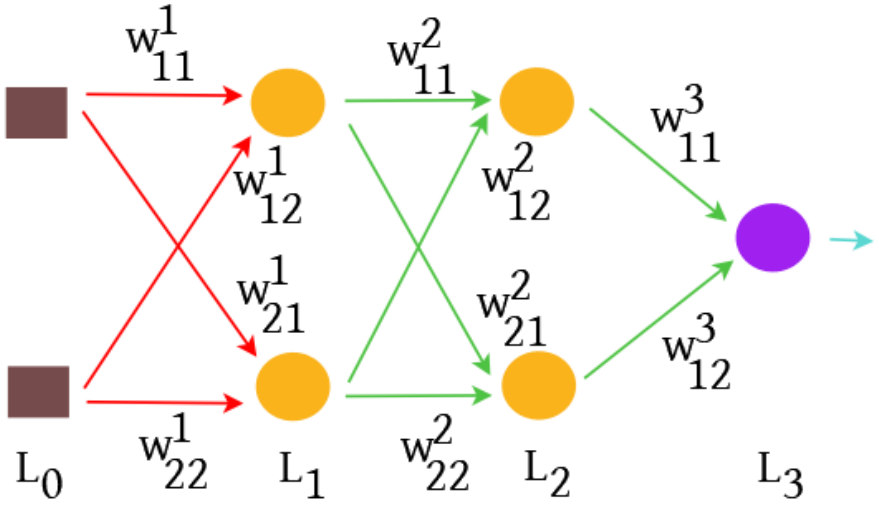
$$\hat{y} = \phi(W^3\phi(W^2\phi(W^1x))) =: \text{MLP}(x)$$

Note: The same activation function ϕ was assumed for simplicity. Typically different activations functions are used for different layers. Then we can write:

$$\hat{y} = \phi_3(W^3\phi_2(W^2\phi_1(W^1x))) =: \text{MLP}(x)$$

where ϕ_1, ϕ_2 and ϕ_3 are activation functions for layers L_1, L_2 and L_3 respectively.

Multi Layer Perceptron - Data Perspective



Given data (x, y), multi layer perceptron predicts:

$$\hat{y} = \phi(W^3\phi(W^2\phi(W^1x))) =: \text{MLP}(x)$$

Similar to perceptron, if $y \neq \hat{y}$ an error $E(y, \hat{y})$ is incurred.